

L08_Diffusion_student_notebook_ITAI_2376 (2)

March 26, 2026

1 Creating Numbers/images with AI: A Hands-on Diffusion Model Exercise

1.1 Introduction

In this assignment, you'll learn how to create an AI model that can generate realistic images from scratch using a powerful technique called 'diffusion'. Think of it like teaching AI to draw by first learning how images get blurry and then learning to make them clear again.

1.1.1 What We'll Build

- A diffusion model capable of generating realistic images
- For most students: An AI that generates handwritten digits (0-9) using the MNIST dataset
- For students with more computational resources: Options to work with more complex datasets
- Visual demonstrations of how random noise gradually transforms into clear, recognizable images
- By the end, your AI should create images realistic enough for another AI to recognize them

1.1.2 Dataset Options

This lab offers flexibility based on your available computational resources:

- Standard Option (Free Colab): We'll primarily use the MNIST handwritten digit dataset, which works well with limited GPU memory and completes training in a reasonable time frame. Most examples and code in this notebook are optimized for MNIST.
- Advanced Option: If you have access to more powerful GPUs (either through Colab Pro/Pro+ or your own hardware), you can experiment with more complex datasets like Fashion-MNIST, CIFAR-10, or even face generation. You'll need to adapt the model architecture, hyperparameters, and evaluation metrics accordingly.

1.1.3 Resource Requirements

- Basic MNIST: Works with free Colab GPUs (2-4GB VRAM), ~30 minutes training
- Fashion-MNIST: Similar requirements to MNIST CIFAR-10: Requires more memory (8-12GB VRAM) and longer training (~2 hours)
- Higher resolution images: Requires substantial GPU resources and several hours of training

1.1.4 Before You Start

1. Make sure you're running this in Google Colab or another environment with GPU access
2. Go to 'Runtime' → 'Change runtime type' and select 'GPU' as your hardware accelerator
3. Each code cell has comments explaining what it does
4. Don't worry if you don't understand every detail - focus on the big picture!
5. If working with larger datasets, monitor your GPU memory usage carefully

The concepts you learn with MNIST will scale to more complex datasets, so even if you're using the basic option, you'll gain valuable knowledge about generative AI that applies to more advanced applications.

1.2 Step 1: Setting Up Our Tools

First, let's install and import all the tools we need. Run this cell and wait for it to complete.

```
[ ]: # torch.cuda.empty_cache()
      # import gc
      # gc.collect()
```

```
[ ]: !pip install ipynbname
      # verbose=False
```

Collecting ipynbname

Downloading ipynbname-2025.8.0.0-py3-none-any.whl.metadata (2.2 kB)
Requirement already satisfied: ipykernel in /usr/local/lib/python3.12/dist-packages (from ipynbname) (6.17.1)
Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (1.8.15)
Requirement already satisfied: ipython>=7.23.1 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (7.34.0)
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (7.4.9)
Requirement already satisfied: matplotlib-inline>=0.1 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (0.2.1)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (1.6.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (26.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (5.9.5)
Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (26.2.1)
Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (6.5.1)
Requirement already satisfied: traitlets>=5.1.0 in /usr/local/lib/python3.12/dist-packages (from ipykernel->ipynbname) (5.7.1)
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3.12/dist-packages (from

```

ipython>=7.23.1->ipykernel->ipynbname) (75.2.0)
Collecting jedi>=0.16 (from ipython>=7.23.1->ipykernel->ipynbname)
  Downloading jedi-0.19.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->ipynbname) (4.4.2)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->ipynbname) (0.7.5)
Requirement already satisfied: prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from
ipython>=7.23.1->ipykernel->ipynbname) (3.0.52)
Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->ipynbname) (2.19.2)
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->ipynbname) (0.2.0)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-
packages (from ipython>=7.23.1->ipykernel->ipynbname) (4.9.0)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-
packages (from jupyter-client>=6.1.12->ipykernel->ipynbname) (0.4)
Requirement already satisfied: jupyter-core>=4.9.2 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->ipykernel->ipynbname) (5.9.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->ipykernel->ipynbname) (2.9.0.post0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.12/dist-packages (from
jedi>=0.16->ipython>=7.23.1->ipykernel->ipynbname) (0.8.6)
Requirement already satisfied: platformdirs>=2.5 in
/usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.9.2->jupyter-
client>=6.1.12->ipykernel->ipynbname) (4.9.4)
Requirement already satisfied: ptyprocess>=0.5 in
/usr/local/lib/python3.12/dist-packages (from
pexpect>4.3->ipython>=7.23.1->ipykernel->ipynbname) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
packages (from prompt-
toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0->ipython>=7.23.1->ipykernel->ipynbname)
(0.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.8.2->jupyter-
client>=6.1.12->ipykernel->ipynbname) (1.17.0)
Downloading ipynbname-2025.8.0.0-py3-none-any.whl (4.5 kB)
Downloading jedi-0.19.2-py2.py3-none-any.whl (1.6 MB)
1.6/1.6 MB
72.2 MB/s eta 0:00:00
Installing collected packages: jedi, ipynbname
Successfully installed ipynbname-2025.8.0.0 jedi-0.19.2

```

```
[ ]: # Step 1: Install required packages
%pip install einops
print("Package installation complete.")

# Step 2: Import libraries
# --- Core PyTorch libraries ---
import numpy as np
import torch # Main deep learning framework
import torch.nn.functional as F # Neural network functions like activation
    ↪ functions
import torch.nn as nn # Neural network building blocks (layers)
from torch.optim import Adam # Optimization algorithm for training

# --- Import the dataset ---
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# --- Data handling ---
from torch.utils.data import Dataset, DataLoader # For organizing and loading
    ↪ our data
import torchvision # Library for computer vision datasets and models
import torchvision.transforms as transforms # For preprocessing images

# --- Tensor manipulation ---
import random # For random operations
from einops.layers.torch import Rearrange # For reshaping tensors in neural
    ↪ networks
from einops import rearrange # For elegant tensor reshaping operations
import numpy as np # For numerical operations on arrays

# --- System utilities ---
import os # For operating system interactions (used for CPU count)

# --- Visualization tools ---
import matplotlib.pyplot as plt # For plotting images and graphs
from PIL import Image # For image processing
from torchvision.utils import save_image, make_grid # For saving and
    ↪ displaying image grids

# Step 3: Set up device (GPU or CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"We'll be using: {device}")

# Check if we're actually using GPU (for students to verify)
if device.type == "cuda":
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
```

```

    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:
↪.2f} GB")
else:
    print("Note: Training will be much slower on CPU. Consider using Google↪
↪Colab with GPU enabled.")

```

Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (0.8.2)

Package installation complete.

We'll be using: cuda

GPU name: Tesla T4

GPU memory: 15.64 GB

1.2.1 REPRODUCIBILITY AND DEVICE SETUP

```

[ ]: # Step 4: Set random seeds for reproducibility
# Diffusion models are sensitive to initialization, so reproducible results↪
↪help with debugging
SEED = 42 # Universal seed value for reproducibility
torch.manual_seed(SEED) # PyTorch random number generator
np.random.seed(SEED) # NumPy random number generator
random.seed(SEED) # Python's built-in random number generator

print(f"Random seeds set to {SEED} for reproducible results")

# Configure CUDA for GPU operations if available
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED) # GPU random number generator
    torch.cuda.manual_seed_all(SEED) # All GPUs random number generator

    # Ensure deterministic GPU operations
    # Note: This slightly reduces performance but ensures results are↪
↪reproducible
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    try:
        # Check available GPU memory
        gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9 #↪
↪Convert to GB
        print(f"Available GPU Memory: {gpu_memory:.1f} GB")

        # Add recommendation based on memory
        if gpu_memory < 4:
            print("Warning: Low GPU memory. Consider reducing batch size if you↪
↪encounter OOM errors.")
        except Exception as e:

```

```

        print(f"Could not check GPU memory: {e}")
else:
    print("No GPU detected. Training will be much slower on CPU.")
    print("If you're using Colab, go to Runtime > Change runtime type and ↵
↵select GPU.")

```

Random seeds set to 42 for reproducible results

Available GPU Memory: 15.6 GB

1.3 Step 2: Choosing Your Dataset

You have several options for this exercise, depending on your computer's capabilities:

1.3.1 Option 1: MNIST (Basic - Works on Free Colab)

- Content: Handwritten digits (0-9)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You're using free Colab or have a basic GPU

1.3.2 Option 2: Fashion-MNIST (Intermediate)

- Content: Clothing items (shirts, shoes, etc.)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You want more interesting images but have limited GPU

1.3.3 Option 3: CIFAR-10 (Advanced)

- Content: Real-world objects (cars, animals, etc.)
- Image size: 32x32 pixels, Color (RGB)
- Training samples: 50,000
- Memory needed: ~4GB GPU
- Training time: ~1-2 hours on Colab
- **Choose this if:** You have Colab Pro or a good local GPU (8GB+ memory)

1.3.4 Option 4: CelebA (Expert)

- Content: Celebrity face images
- Image size: 64x64 pixels, Color (RGB)
- Training samples: 200,000
- Memory needed: ~8GB GPU
- Training time: ~3-4 hours on Colab
- **Choose this if:** You have excellent GPU (12GB+ memory)

To use your chosen dataset, uncomment its section in the code below and make sure all others are commented out.

```
[ ]: #=====
# SECTION 2: DATASET SELECTION AND CONFIGURATION
#=====
# STUDENT INSTRUCTIONS:
# 1. Choose ONE dataset option based on your available GPU memory
# 2. Uncomment ONLY ONE dataset section below
# 3. Make sure all other dataset sections remain commented out

#-----
# OPTION 1: MNIST (Basic - 2GB GPU)
#-----
# Recommended for: Free Colab or basic GPU
# Memory needed: ~2GB GPU
# Training time: ~15-30 minutes

IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 32
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the MNIST dataset
# Hint: Use torchvision.datasets.MNIST with root='./data', train=True,
#       transform=transform, and download=True
# Then print a success message

# Enter your code here:
# Load the MNIST dataset
train_dataset = datasets.MNIST(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

test_dataset = datasets.MNIST(
    root='./data',
    train=False,
    transform=transform,
    download=True
)
```

```

)

# Create data loaders
train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2
)

print(f" Successfully loaded MNIST dataset!")
print(f" Training samples: {len(train_dataset)}")
print(f" Test samples: {len(test_dataset)}")
print(f" Image size: {IMG_SIZE}x{IMG_SIZE}, Channels: {IMG_CH}")
print(f" Number of classes: {N_CLASSES}")
print(f" Batch size: {BATCH_SIZE}")

#-----
# OPTION 2: Fashion-MNIST (Intermediate - 2GB GPU)
#-----
# Uncomment this section to use Fashion-MNIST instead
"""
IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the Fashion-MNIST dataset
# Hint: Very similar to MNIST but use torchvision.datasets.FashionMNIST

# Enter your code here:

"""

```



```

#-----
# OPTION 3: CIFAR-10 (Advanced - 4GB+ GPU)
#-----
# Uncomment this section to use CIFAR-10 instead
"""
IMG_SIZE = 32
IMG_CH = 3
N_CLASSES = 10
BATCH_SIZE = 32 # Reduced batch size for memory
EPOCHS = 50      # More epochs for complex data

# Your code to create the transform and load CIFAR-10
# Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.
    ↪5, 0.5, 0.5))
# Then load torchvision.datasets.CIFAR10

# Enter your code here:

"""

```

```

100%|      | 9.91M/9.91M [00:01<00:00, 5.59MB/s]
100%|      | 28.9k/28.9k [00:00<00:00, 136kB/s]
100%|      | 1.65M/1.65M [00:01<00:00, 1.26MB/s]
100%|      | 4.54k/4.54k [00:00<00:00, 12.3MB/s]

```

```

Successfully loaded MNIST dataset!
Training samples: 60000
Test samples: 10000
Image size: 28x28, Channels: 1
Number of classes: 10
Batch size: 32

```

```

[ ]: '\nIMG_SIZE = 32\nIMG_CH = 3\nN_CLASSES = 10\nBATCH_SIZE = 32 # Reduced batch
size for memory\nEPOCHS = 50      # More epochs for complex data\n\n# Your code
to create the transform and load CIFAR-10\n# Hint: Use transforms.Normalize with
RGB means and stds ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))\n# Then load
torchvision.datasets.CIFAR10\n\n# Enter your code here:\n\n'

```

```

[ ]: #Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'train_dataset' not in locals():
    raise ValueError("""
        ERROR: No dataset selected! Please uncomment exactly one dataset option.
    """)

```

```

    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    """)

# Validate GPU memory requirements
print("=" * 60)
print("GPU Memory Validation")
print("=" * 60)

# Check if CUDA is available
if torch.cuda.is_available():
    # Get GPU properties
    device_props = torch.cuda.get_device_properties(0)
    total_memory_gb = device_props.total_memory / (1024**3) # Convert bytes to GB
    gpu_name = device_props.name

    print(f" GPU Available: {gpu_name}")
    print(f" Total GPU Memory: {total_memory_gb:.2f} GB")

# Define memory requirements for each dataset
memory_requirements = {
    'MNIST': 2.0,
    'Fashion-MNIST': 2.0,
    'CIFAR-10': 4.0
}

# Determine which dataset is being used based on IMG_SIZE and IMG_CH
if IMG_SIZE == 28 and IMG_CH == 1:
    if 'FashionMNIST' in str(type(train_dataset)):
        dataset_name = 'Fashion-MNIST'
    else:
        dataset_name = 'MNIST'
elif IMG_SIZE == 32 and IMG_CH == 3:
    dataset_name = 'CIFAR-10'
else:
    dataset_name = 'Unknown'

required_memory = memory_requirements.get(dataset_name, 2.0)

print(f" Dataset Selected: {dataset_name}")
print(f" Required Memory: {required_memory:.1f} GB")

# Check if GPU has enough memory
if total_memory_gb >= required_memory:

```

```

        print(f" GPU memory is sufficient for {dataset_name}!")
        print(f"    Available: {total_memory_gb:.2f} GB >= Required:␣
↪{required_memory:.1f} GB")
    else:
        print(f"    WARNING: GPU memory may be insufficient!")
        print(f"    Available: {total_memory_gb:.2f} GB < Required:␣
↪{required_memory:.1f} GB")
        print(f"    Consider:")
        print(f"    - Reducing BATCH_SIZE")
        print(f"    - Using a simpler dataset (MNIST)")
        print(f"    - Using a machine with more GPU memory")
else:
    print(" No GPU detected - training will use CPU")
    print(" This will be MUCH slower for diffusion models.")
    print(" Consider using Google Colab with GPU runtime.")

print("=" * 60)

```

=====

GPU Memory Validation

=====

```

GPU Available: Tesla T4
Total GPU Memory: 14.56 GB
Dataset Selected: MNIST
Required Memory: 2.0 GB
GPU memory is sufficient for MNIST!
Available: 14.56 GB >= Required: 2.0 GB
=====

```

```

[ ]: # #Validating Dataset Selection
# #Let's add code to validate that a dataset was selected
# # and check if your GPU has enough memory:

# # Validate dataset selection
# if 'dataset' not in locals():
#     raise ValueError("""
#         ERROR: No dataset selected! Please uncomment exactly one dataset option.
#     Available options:
#     1. MNIST (Basic) - 2GB GPU
#     2. Fashion-MNIST (Intermediate) - 2GB GPU
#     3. CIFAR-10 (Advanced) - 4GB+ GPU
#     4. CelebA (Expert) - 8GB+ GPU
#     """)

# # Your code to validate GPU memory requirements
# # Hint: Check torch.cuda.is_available() and use torch.cuda.
↪get_device_properties(0).total_memory

```

```
# # to get available GPU memory, then compare with dataset requirements

# # Enter your code here:
```

```
[ ]: #Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Check sample batch properties
print("=" * 60)
print("Dataset Properties")
print("=" * 60)

# Get a sample batch
sample_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
sample_images, sample_labels = next(iter(sample_loader))

print(f"Sample batch shape: {sample_images.shape}")
print(f" - Batch size: {sample_images.shape[0]}")
print(f" - Channels: {sample_images.shape[1]}")
print(f" - Height: {sample_images.shape[2]}")
print(f" - Width: {sample_images.shape[3]}")
print(f"\nData type: {sample_images.dtype}")
print(f"Value range: [{sample_images.min():.3f}, {sample_images.max():.3f}]")
print(f"Mean: {sample_images.mean():.3f}")
print(f"Std: {sample_images.std():.3f}")
print(f"\nLabel: {sample_labels.item()} (class: {sample_labels.item()})")
print(f"Label type: {sample_labels.dtype}")

# Show a few more samples
print(f"\nTotal training samples: {len(train_dataset)}")
print(f"Total test samples: {len(test_dataset)}")
print("=" * 60)

#=====
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#=====
# Create train-validation split

print("\n" + "=" * 60)
print("Creating Train-Validation Split")
print("=" * 60)

from torch.utils.data import random_split

# Calculate split sizes (80% train, 20% validation)
```

```

train_size = int(0.8 * len(train_dataset))
val_size = len(train_dataset) - train_size

# Create reproducible split using fixed seed
generator = torch.Generator().manual_seed(42)
train_subset, val_subset = random_split(
    train_dataset,
    [train_size, val_size],
    generator=generator
)

print(f"Original training set size: {len(train_dataset)}")
print(f"New training set size: {len(train_subset)} (80%)")
print(f"Validation set size: {len(val_subset)} (20%)")
print(f"Test set size: {len(test_dataset)}")

# Create dataloaders for training and validation
print("\n" + "=" * 60)
print("Creating DataLoaders")
print("=" * 60)

import os

# Determine number of workers based on CPU cores
num_workers = min(4, os.cpu_count() or 2) # Use max 4 workers or available CPUs

# Training dataloader (with shuffling)
train_loader = DataLoader(
    train_subset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=num_workers,
    pin_memory=True if torch.cuda.is_available() else False
)

# Validation dataloader (no shuffling)
val_loader = DataLoader(
    val_subset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=True if torch.cuda.is_available() else False
)

# Test dataloader (no shuffling)
test_loader = DataLoader(

```

```

    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=True if torch.cuda.is_available() else False
)

print(f"Batch size: {BATCH_SIZE}")
print(f"Number of workers: {num_workers}")
print(f"Pin memory: {torch.cuda.is_available()}")
print(f"\nDataLoader created successfully!")
print(f" - Training batches: {len(train_loader)}")
print(f" - Validation batches: {len(val_loader)}")
print(f" - Test batches: {len(test_loader)}")
print("=" * 60)

```

```

=====
Dataset Properties
=====
Sample batch shape: torch.Size([1, 1, 28, 28])
  - Batch size: 1
  - Channels: 1
  - Height: 28
  - Width: 28

Data type: torch.float32
Value range: [-1.000, 0.992]
Mean: -0.850
Std: 0.478

Label: 1 (class: 1)
Label type: torch.int64

Total training samples: 60000
Total test samples: 10000
=====

=====
Creating Train-Validation Split
=====
Original training set size: 60000
New training set size: 48000 (80%)
Validation set size: 12000 (20%)
Test set size: 10000

=====
Creating DataLoaders

```

```
=====
Batch size: 32
Number of workers: 2
Pin memory: True
```

```
DataLoader created successfully!
- Training batches: 1500
- Validation batches: 375
- Test batches: 313
=====
```

```
[ ]: #Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Your code to check sample batch properties
# Hint: Get a sample batch using next(iter(DataLoader(dataset, batch_size=1)))
# Then print information about the dataset shape, type, and value ranges

# Enter your code here:

#=====
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#=====
# Create train-validation split

# Your code to create a train-validation split (80% train, 20% validation)
# Hint: Use random_split() with appropriate train_size and val_size
# Be sure to use a fixed generator for reproducibility

# Enter your code here:

# Your code to create dataloaders for training and validation
# Hint: Use DataLoader with batch_size=BATCH_SIZE, appropriate shuffle settings,
# and num_workers based on available CPU cores

# Enter your code here:
```

1.4 Step 3: Building Our Model Components

Now we'll create the building blocks of our AI model. Think of these like LEGO pieces that we'll put together to make our number generator:

- GELUConvBlock: The basic building block that processes images
- DownBlock: Makes images smaller while finding important features
- UpBlock: Makes images bigger again while keeping the important features

- Other blocks: Help the model understand time and what number to generate

```
[ ]: # Basic building block that processes images
class GELUConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, group_size):
        """
        Creates a block with convolution, normalization, and activation

        Args:
            in_ch (int): Number of input channels
            out_ch (int): Number of output channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Check that group_size is compatible with out_ch
        if out_ch % group_size != 0:
            print(f"Warning: out_ch ({out_ch}) is not divisible by group_size_
↳ ({group_size})")
            # Adjust group_size to be compatible
            group_size = min(group_size, out_ch)
            while out_ch % group_size != 0:
                group_size -= 1
            print(f"Adjusted group_size to {group_size}")

        # Create layers for the block
        self.model = nn.Sequential(
            # Convolutional layer with 3x3 kernel and padding=1 to preserve_
↳ spatial dimensions
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),

            # Group normalization for stable training
            nn.GroupNorm(num_groups=group_size, num_channels=out_ch),

            # GELU activation (smoother than ReLU, good for diffusion models)
            nn.GELU()
        )

    def forward(self, x):
        # Forward pass through the sequential model
        return self.model(x)
```

```
[ ]: from einops.layers.torch import Rearrange

# Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
class RearrangePoolBlock(nn.Module):
    def __init__(self, in_chs, group_size):
```



```

    """
    Downsamples the spatial dimensions by 2x while preserving information

    Args:
        in_chs (int): Number of input channels
        group_size (int): Number of groups for GroupNorm
    """
    super().__init__()

    # Rearrange operation: combines 2x2 pixel blocks into channels
    # This reduces spatial dimensions (H, W) by 2x while increasing
    ↪ channels by 4x
    # Example: (B, C, 28, 28) -> (B, C*4, 14, 14)
    self.rearrange = Rearrange('b c (h p1) (w p2) -> b (c p1 p2) h w',
    ↪ p1=2, p2=2)

    # Convolution to process the rearranged tensor
    # Input channels are multiplied by 4 (2x2 pixels become 4 channels)
    self.conv = GELUConvBlock(in_chs * 4, in_chs, group_size)

    def forward(self, x):
        # Apply rearrange to downsample spatial dimensions
        x = self.rearrange(x)

        # Apply convolution to process rearranged features
        x = self.conv(x)

    return x

```

```

[ ]: #Let's implement the upsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__() # Simplified super() call, equivalent to original

        # Sequential processing of features

```

```

        layers = [
            GELUConvBlock(in_chs, out_chs, group_size), # First conv block
            ↪changes channel dimensions
            GELUConvBlock(out_chs, out_chs, group_size), # Second conv block
            ↪processes features
            RearrangePoolBlock(out_chs, group_size) # Downsampling
            ↪(spatial dims: H,W → H/2,W/2)
        ]
        self.model = nn.Sequential(*layers)

        # Log the configuration for debugging
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs},
            ↪spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """
        return self.model(x)

```

```

[ ]: class UpBlock(nn.Module):
    """
    Upsampling block for decoding path in U-Net architecture.

    This block:
    1. Takes features from the decoding path and corresponding skip connection
    2. Concatenates them along the channel dimension
    3. Upsamples spatial dimensions by 2x using transposed convolution
    4. Processes features through multiple convolutional blocks

    Args:
        in_chs (int): Number of input channels from the previous layer
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__()

        # Upsample first, THEN cat with skip
        self.upsample = nn.ConvTranspose2d(
            in_channels=in_chs, # just in_chs, no concatenation yet

```

```

        out_channels=out_chs,
        kernel_size=2,
        stride=2
    )

    # After upsample: out_chs channels
    # After cat with skip: out_chs + out_chs channels
    self.conv1 = GELUConvBlock(out_chs + out_chs, out_chs,
↪group_size=group_size)
    self.conv2 = GELUConvBlock(out_chs, out_chs, group_size=group_size)

    def forward(self, x, skip):
        # Step 1: Upsample FIRST
        x = self.upsample(x)

        # Step 2: Align spatial dims if needed
        if x.shape[2:] != skip.shape[2:]:
            x = F.interpolate(x, size=skip.shape[2:], mode='bilinear',
↪align_corners=False)

        # Step 3: THEN concatenate with skip
        x = torch.cat([x, skip], dim=1) # out_chs + out_chs channels

        # Step 4: Process
        x = self.conv1(x)
        x = self.conv2(x)

    return x

```

```

[ ]: # # del UpBlock

# class UpBlock(nn.Module):
#     def __init__(self, in_chs, out_chs, group_size):
#         super().__init__()
#         self.upsample = nn.ConvTranspose2d(in_chs, out_chs, kernel_size=2,
↪stride=2)
#         self.conv1 = GELUConvBlock(out_chs + out_chs, out_chs,
↪group_size=group_size)
#         self.conv2 = GELUConvBlock(out_chs, out_chs, group_size=group_size)

#     def forward(self, x, skip):
#         x = self.upsample(x)
#         if x.shape[2:] != skip.shape[2:]:
#             x = F.interpolate(x, size=skip.shape[2:], mode='bilinear',
↪align_corners=False)
#         x = torch.cat([x, skip], dim=1)
#         x = self.conv1(x)

```

```

#         x = self.conv2(x)
#         return x

# # Rebuild model in the SAME cell
# model = UNet(
#     T=n_steps,
#     img_ch=IMG_CH,
#     img_size=IMG_SIZE,
#     down_chs=(32, 64, 128),
#     t_embed_dim=128,
#     c_embed_dim=N_CLASSES
# ).to(device)

# # Verify
# for i, up in enumerate(model.ups):
#     for name, param in up.named_parameters():
#         if 'weight' in name:
#             print(f"ups.{i} {name}: {param.shape}")

```

```

[ ]: import dis
dis.dis(UpBlock.forward)

```

32	0 RESUME	0
34	2 LOAD_FAST	0 (self)
	4 LOAD_ATTR	1 (NULL self + upsample)
	24 LOAD_FAST	1 (x)
	26 CALL	1
	34 STORE_FAST	1 (x)
37	36 LOAD_FAST	1 (x)
	38 LOAD_ATTR	2 (shape)
	58 LOAD_CONST	1 (2)
	60 LOAD_CONST	0 (None)
	62 BINARY_SLICE	
	64 LOAD_FAST	2 (skip)
	66 LOAD_ATTR	2 (shape)
	86 LOAD_CONST	1 (2)
	88 LOAD_CONST	0 (None)
	90 BINARY_SLICE	
	92 COMPARE_OP	55 (!=)
	96 POP_JUMP_IF_FALSE	38 (to 174)
38	98 LOAD_GLOBAL	4 (F)
	108 LOAD_ATTR	7 (NULL self + interpolate)
	128 LOAD_FAST	1 (x)
	130 LOAD_FAST	2 (skip)

```

132 LOAD_ATTR          2 (shape)
152 LOAD_CONST         1 (2)
154 LOAD_CONST         0 (None)
156 BINARY_SLICE
158 LOAD_CONST         2 ('bilinear')
160 LOAD_CONST         3 (False)
162 KW_NAMES           4 (('size', 'mode', 'align_corners'))
164 CALL               4
172 STORE_FAST        1 (x)

41    >> 174 LOAD_GLOBAL      8 (torch)
184 LOAD_ATTR         11 (NULL|self + cat)
204 LOAD_FAST         1 (x)
206 LOAD_FAST         2 (skip)
208 BUILD_LIST        2
210 LOAD_CONST        5 (1)
212 KW_NAMES          6 (('dim',))
214 CALL              2
222 STORE_FAST        1 (x)

44    224 LOAD_FAST         0 (self)
226 LOAD_ATTR         13 (NULL|self + conv1)
246 LOAD_FAST         1 (x)
248 CALL              1
256 STORE_FAST        1 (x)

45    258 LOAD_FAST         0 (self)
260 LOAD_ATTR         15 (NULL|self + conv2)
280 LOAD_FAST         1 (x)
282 CALL              1
290 STORE_FAST        1 (x)

47    292 LOAD_FAST         1 (x)
294 RETURN_VALUE

```

```

[ ]: # #Now let's implement the upsampling block for our U-Net architecture:
# class UpBlock(nn.Module):
#     """
#     Upsampling block for decoding path in U-Net architecture.
#
#     This block:
#     1. Takes features from the decoding path and corresponding skip connection
#     2. Concatenates them along the channel dimension
#     3. Upsamples spatial dimensions by 2x using transposed convolution
#     4. Processes features through multiple convolutional blocks
#
#     Args:

```

```

#         in_chs (int): Number of input channels from the previous layer
#         out_chs (int): Number of output channels
#         group_size (int): Number of groups for GroupNorm
#         """
#         def __init__(self, in_chs, out_chs, group_size):
#             super().__init__()

#         # Transposed convolution for upsampling
#         # Input channels are 2 * in_chs because we concatenate skip connection
#         # kernel_size=2 and stride=2 doubles spatial dimensions (H, W) ->
#         ↪ (2H, 2W)
#         self.upsample = nn.ConvTranspose2d(
#             in_channels=in_chs, # Concatenated features
#             out_channels=out_chs,
#             kernel_size=2,
#             stride=2
#         )

#         # Convolutional blocks to process upsampled features
#         # Using multiple blocks helps refine the features after upsampling
#         self.conv1 = GELUConvBlock(out_chs + out_chs, out_chs,
#         ↪ group_size=group_size) # after cat with skip
#         self.conv2 = GELUConvBlock(out_chs, out_chs, group_size=group_size)
#         # self.conv1 = GELUConvBlock(out_chs, out_chs, group_size)
#         # self.conv2 = GELUConvBlock(out_chs, out_chs, group_size)

#         # Log the configuration for debugging
#         print(f"Created UpBlock: in_chs={in_chs}, out_chs={out_chs},
#         ↪ spatial_increase=2x")

#         def forward(self, x, skip):
#             """
#             Forward pass through the UpBlock.

#             Args:
#             x (torch.Tensor): Input tensor from previous layer [B, in_chs, H,
#             ↪ W]
#             skip (torch.Tensor): Skip connection tensor from encoder [B,
#             ↪ in_chs, 2H, 2W]

#             Returns:
#             torch.Tensor: Output tensor with shape [B, out_chs, 2H, 2W]
#             """
#             # Step 1: Concatenate input with skip connection along channel
#             ↪ dimension

```

```

#           # x: [B, in_chs, H, W] + skip: [B, in_chs, H, W] -> [B, 2*in_chs, H, W]
#           ↪W]
#           x = torch.cat([x, skip], dim=1)

#           # Step 2: Upsample using transposed convolution
#           # [B, 2*in_chs, H, W] -> [B, out_chs, 2H, 2W]
#           x = self.upsample(x)

#           # Step 3: Process through convolutional blocks
#           # [B, out_chs, 2H, 2W] -> [B, out_chs, 2H, 2W]
#           x = self.conv1(x)
#           x = self.conv2(x)

#           return x

```

```

[ ]: # Here we implement the time embedding block for our U-Net architecture:
# Helps the model understand time steps in diffusion process
class SinusoidalPositionEmbedBlock(nn.Module):
    """
    Creates sinusoidal embeddings for time steps in diffusion process.

    This embedding scheme is adapted from the Transformer architecture and
    provides a unique representation for each time step that preserves
    relative distance information.

    Args:
        dim (int): Embedding dimension
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, time):
        """
        Computes sinusoidal embeddings for given time steps.

        Args:
            time (torch.Tensor): Time steps tensor of shape [batch_size]

        Returns:
            torch.Tensor: Time embeddings of shape [batch_size, dim]
        """
        device = time.device
        half_dim = self.dim // 2
        embeddings = torch.log(torch.tensor(10000.0, device=device)) / ↪
        ↪(half_dim - 1)

```

```

        embeddings = torch.exp(torch.arange(half_dim, device=device) *  $\frac{\pi}{2}$ 
↪-embeddings)
        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings

```

```

[ ]: # Helps the model understand which number/image to draw (class conditioning)
class EmbedBlock(nn.Module):
    """
    Creates embeddings for class conditioning in diffusion models.

    This module transforms a one-hot or index representation of a class
    into a rich embedding that can be added to feature maps.

    Args:
        input_dim (int): Input dimension (typically number of classes)
        emb_dim (int): Output embedding dimension
    """
    def __init__(self, input_dim, emb_dim):
        super(EmbedBlock, self).__init__()
        self.input_dim = input_dim

        # Create the embedding layers using sequential model
        self.model = nn.Sequential(
            # First linear layer: projects from input_dim to emb_dim
            nn.Linear(input_dim, emb_dim),

            # GELU activation for non-linearity
            nn.GELU(),

            # Second linear layer: refines the embedding
            nn.Linear(emb_dim, emb_dim),

            # Unflatten to add spatial dimensions for broadcasting
            # [batch, emb_dim] -> [batch, emb_dim, 1, 1]
            # This allows adding embeddings to feature maps of any spatial size
            nn.Unflatten(dim=1, unflattened_size=(emb_dim, 1, 1))
        )

    def forward(self, x):
        """
        Computes class embeddings for the given class indices.

        Args:
            x (torch.Tensor): Class indices or one-hot encodings [batch_size,  $\frac{1}{2}$ 
↪input_dim]

```



```

Returns:
    torch.Tensor: Class embeddings of shape [batch_size, emb_dim, 1, 1]
                  (ready to be added to feature maps)
"""
x = x.view(-1, self.input_dim)
return self.model(x)

```

```

[ ]: import torch
import torch.nn.functional as F
from torch import nn
from einops.layers.torch import Rearrange
from einops import rearrange

# Main U-Net model that puts everything together
class UNet(nn.Module):
    """
    U-Net architecture for diffusion models with time and class conditioning.

    This architecture follows the standard U-Net design with:
    1. Downsampling path that reduces spatial dimensions
    2. Middle processing blocks
    3. Upsampling path that reconstructs spatial dimensions
    4. Skip connections between symmetric layers

    The model is conditioned on:
    - Time step (where we are in the diffusion process)
    - Class labels (what we want to generate)

    Args:
        T (int): Number of diffusion time steps
        img_ch (int): Number of image channels
        img_size (int): Size of input images
        down_chs (list): Channel dimensions for each level of U-Net
        t_embed_dim (int): Dimension for time embeddings
        c_embed_dim (int): Dimension for class embeddings
    """
    def __init__(self, T, img_ch, img_size, down_chs, t_embed_dim, c_embed_dim):
        super().__init__()

        # Time embedding - encodes which diffusion step we're at
        self.time_embed = nn.Sequential(
            SinusoidalPositionEmbedBlock(t_embed_dim),
            nn.Linear(t_embed_dim, down_chs[-1]),
            nn.GELU(),
            nn.Linear(down_chs[-1], down_chs[-1])
        )

```

```

# Class embedding - encodes which class we want to generate
self.class_embed = EmbedBlock(c_embed_dim, down_chs[-1])

# Initial convolution to process input image
self.init_conv = GELUConvBlock(img_ch, down_chs[0], group_size=8)

# Downsampling path - reduces spatial dimensions, increases channels
self.downs = nn.ModuleList()
in_ch = down_chs[0]
for out_ch in down_chs[1:]:
    self.downs.append(DownBlock(in_ch, out_ch, group_size=8))
    in_ch = out_ch

# Middle blocks - process features at lowest resolution
self.mid1 = GELUConvBlock(down_chs[-1], down_chs[-1], group_size=8)
self.mid2 = GELUConvBlock(down_chs[-1], down_chs[-1], group_size=8)

# Upsampling path - reconstructs spatial dimensions
self.ups = nn.ModuleList()
reversed_chs = list(reversed(down_chs))
for i in range(len(reversed_chs) - 1):
    self.ups.append(UpBlock(reversed_chs[i], reversed_chs[i+1],
↪group_size=8))

# Final convolution to project back to image space
self.final_conv = nn.Conv2d(down_chs[0], img_ch, kernel_size=1)

print(f"Created UNet with {len(down_chs)} scale levels")
print(f"Channel dimensions: {down_chs}")

def forward(self, x, t, c, c_mask):
    """
    Forward pass through the UNet.

    Args:
        x (torch.Tensor): Input noisy image [B, img_ch, H, W]
        t (torch.Tensor): Diffusion time steps [B]
        c (torch.Tensor): Class labels [B, c_embed_dim]
        c_mask (torch.Tensor): Mask for conditional generation [B, 1]

    Returns:
        torch.Tensor: Predicted noise in the input image [B, img_ch, H, W]
    """

    # Process time embedding
    # t: [B] -> [B, t_embed_dim] -> [B, down_chs[0], 1, 1]
    t_emb = self.time_embed(t)

```

```

# print(f"t_emb after time_embed: {t_emb.shape}") # check this
# t_emb = t_emb.view(t_emb.shape[0], -1, 1, 1)
# print(f"t_emb after view: {t_emb.shape}") # and this
# Correctly reshape t_emb for broadcasting: [B, C] -> [B, C, 1, 1]
t_emb = t_emb.unsqueeze(-1).unsqueeze(-1)

# Process class embedding
# c: [B, c_embed_dim] -> [B, down_chs[0], 1, 1]
c_emb = self.class_embed(c)

# Apply class mask for classifier-free guidance
# When c_mask=0, we zero out the class embedding (unconditional)
c_emb = c_emb * c_mask.view(-1, 1, 1, 1)

# Initial feature extraction
# x: [B, img_ch, H, W] -> [B, down_chs[0], H, W]
x = self.init_conv(x)

# Downsampling path with skip connections
skip_connections = [] # Store features for skip connections

for down in self.downs:
    # x = down(x)
    skip_connections.append(x)
    x = down(x)

# Remove last skip connection (it's at the bottleneck)
# skip_connections.pop()

# Middle processing with time and class conditioning
x = self.mid1(x)
x = x + t_emb + c_emb # Add conditioning
x = self.mid2(x)

# Upsampling path with skip connections
for up in self.ups:
    # Pop skip connection in reverse order
    skip = skip_connections.pop()
    if x.shape[2:] != skip.shape[2:]:
        x = F.interpolate(x, size=skip.shape[2:], mode='bilinear',
↪align_corners=False)
    x = up(x, skip)

# Final projection to image space
# [B, down_chs[0], H, W] -> [B, img_ch, H, W]
x = self.final_conv(x)

```

```

return x
print(f"t_emb after time_embed: {t_emb.shape}")
print(f"t_emb after view: {t_emb.shape}")

```

```

[ ]: # torch.cuda.empty_cache()

# # Test with a tiny single batch
# test_images = torch.randn(2, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)
# test_t = torch.randint(0, n_steps, (2,)).to(device)
# test_c = torch.zeros(2, N_CLASSES).to(device)
# test_mask = torch.ones(2, 1).to(device)

# print("Input shapes:")
# print(f"  x: {test_images.shape}")
# print(f"  t: {test_t.shape}")
# print(f"  c: {test_c.shape}")

# with torch.no_grad():
#     t_emb = model.time_embed(test_t)
#     print(f"t_emb: {t_emb.shape}")

#     c_emb = model.class_embed(test_c)
#     print(f"c_emb: {c_emb.shape}")

#     out = model(test_images, test_t, test_c, test_mask)
#     print(f"output: {out.shape}")

```

```

[ ]: # with torch.no_grad():
#     test_t = torch.randint(0, n_steps, (2,)).to(device)

#     # Step through time_embed layer by layer
#     x = test_t
#     for i, layer in enumerate(model.time_embed):
#         x = layer(x)
#         print(f"time_embed layer {i} ({layer.__class__.__name__}): {x.shape}")

```

```

[ ]: # Debugging cell to find the which step contains the 6D tensor
# with torch.no_grad():
#     test_images = torch.randn(2, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)
#     test_t = torch.randint(0, n_steps, (2,)).to(device)
#     test_c = torch.zeros(2, N_CLASSES).to(device)
#     test_mask = torch.ones(2, 1).to(device)

#     t_emb = model.time_embed(test_t)
#     print(f"1. t_emb: {t_emb.shape}")

#     t_emb = t_emb.view(t_emb.shape[0], -1, 1, 1)

```

```

#     print(f"2. t_emb after view: {t_emb.shape}")

#     c_emb = model.class_embed(test_c)
#     print(f"3. c_emb: {c_emb.shape}")

#     x = model.init_conv(test_images)
#     print(f"4. after init_conv: {x.shape}")

#     skip_connections = []
#     for i, down in enumerate(model.downs):
#         skip_connections.append(x)
#         x = down(x)
#         print(f"5. after down {i}: {x.shape}")

#     x = model.mid1(x)
#     print(f"6. after mid1: {x.shape}")

#     print(f"7. x + t_emb + c_emb: {x.shape} + {t_emb.shape} + {c_emb.shape}")
#     x = x + t_emb + c_emb
#     print(f"8. after conditioning: {x.shape}")

```

1.5 Step 4: Setting Up The Diffusion Process

Now we'll create the process of adding and removing noise from images. Think of it like: 1. Adding fog: Slowly making the image more and more blurry until you can't see it 2. Removing fog: Teaching the AI to gradually make the image clearer 3. Controlling the process: Making sure we can generate specific numbers we want

```

[ ]: # Set up the noise schedule
n_steps = 100 # How many steps to go from clear image to noise
beta_start = 0.0001 # Starting noise level (small)
beta_end = 0.02 # Ending noise level (larger)

# Create schedule of gradually increasing noise levels
beta = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Calculate important values used in diffusion equations
alpha = 1 - beta # Portion of original image to keep at each step
alpha_bar = torch.cumprod(alpha, dim=0) # Cumulative product of alphas
sqrt_alpha_bar = torch.sqrt(alpha_bar) # For scaling the original image
sqrt_one_minus_alpha_bar = torch.sqrt(1 - alpha_bar) # For scaling the noise

[ ]: # Function to add noise to images (forward diffusion process)
def add_noise(x_0, t):
    """
    Add noise to images according to the forward diffusion process.

```

The formula is: $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1 - \alpha_t} * \epsilon$
 where ϵ is random noise and α_t is the cumulative product of $(1 - \beta_t)$.

Args:

x_0 (torch.Tensor): Original clean image [B, C, H, W]
 t (torch.Tensor): Timestep indices indicating noise level [B]

Returns:

tuple: (noisy_image, noise_added)
 - noisy_image is the image with noise added
 - noise_added is the actual noise that was added (for training)

```
"""
# Create random Gaussian noise with same shape as image
noise = torch.randn_like(x_0)

# Get noise schedule values for the specified timesteps
# Reshape to allow broadcasting with image dimensions
sqrt_alpha_bar_t = sqrt_alpha_bar[t].reshape(-1, 1, 1, 1)
sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

# Apply the forward diffusion equation:
#  $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1 - \alpha_t} * \epsilon$ 
# This creates a mixture of the original image (scaled down) and noise
# (scaled up)
x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise

return x_t, noise
```

```
[ ]: # Function to remove noise from images (reverse diffusion process)
@torch.no_grad() # Don't track gradients during sampling (inference only)
def remove_noise(x_t, t, model, c, c_mask):
    """
    Remove noise from images using the learned reverse diffusion process.

    This implements a single step of the reverse diffusion sampling process.
    The model predicts the noise in the image, which we then use to partially
    denoise the image.
```

Args:

x_t (torch.Tensor): Noisy image at timestep t [B, C, H, W]
 t (torch.Tensor): Current timestep indices [B]
 model (nn.Module): U-Net model that predicts noise
 c (torch.Tensor): Class conditioning (what digit to generate) [B, C]
 c_mask (torch.Tensor): Mask for conditional generation [B, 1]

Returns:

```

        torch.Tensor: Less noisy image for the next timestep [B, C, H, W]
        """
        # Predict the noise in the image using our model
        predicted_noise = model(x_t, t, c, c_mask)

        # Get noise schedule values for the current timestep
        alpha_t = alpha[t].reshape(-1, 1, 1, 1)
        alpha_bar_t = alpha_bar[t].reshape(-1, 1, 1, 1)
        beta_t = beta[t].reshape(-1, 1, 1, 1)
        sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1,
↪1)

        # Special case: if we're at the first timestep (t=0), we're done
        if t[0] == 0:
            return x_t
        else:
            # Calculate the mean of the denoised distribution
            # This is derived from Bayes' rule and the diffusion process equations
            mean = (1 / torch.sqrt(alpha_t)) * (
                x_t - (beta_t / sqrt_one_minus_alpha_bar_t) * predicted_noise
            )

            # Add a small amount of random noise (variance depends on timestep)
            # This helps prevent the generation from becoming too deterministic
            noise = torch.randn_like(x_t)

            # Return the partially denoised image with a bit of new random noise
            return mean + torch.sqrt(beta_t) * noise

```

```

[ ]: # Visualization function to show how noise progressively affects images
def show_noise_progression(image, num_steps=5):
    """
    Visualize how an image gets progressively noisier in the diffusion process.

    Args:
        image (torch.Tensor): Original clean image [C, H, W]
        num_steps (int): Number of noise levels to show
    """
    plt.figure(figsize=(15, 3))

    # Show original image
    plt.subplot(1, num_steps, 1)
    if IMG_CH == 1: # Grayscale image
        plt.imshow(image[0].cpu(), cmap='gray')
    else: # Color image
        img = image.permute(1, 2, 0).cpu() # Change from [C,H,W] to [H,W,C]
        if img.min() < 0: # If normalized between -1 and 1
            img = (img + 1) / 2 # Rescale to [0,1] for display

```

```

plt.imshow(img)
plt.title('Original')
plt.axis('off')

# Show progressively noisier versions
for i in range(1, num_steps):
    # Calculate timestep index based on percentage through the process
    t_idx = int((i/num_steps) * n_steps)
    t = torch.tensor([t_idx]).to(device)

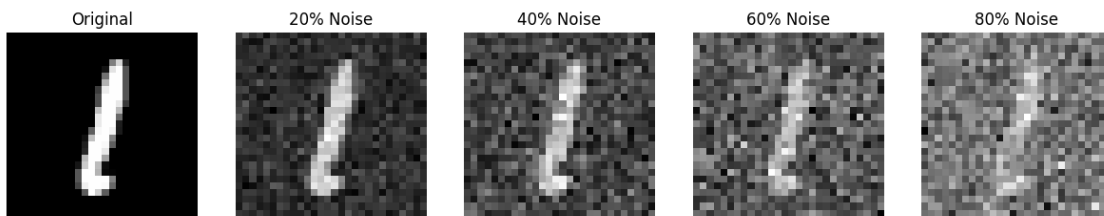
    # Add noise corresponding to timestep t
    noisy_image, _ = add_noise(image.unsqueeze(0), t)

    # Display the noisy image
    plt.subplot(1, num_steps, i+1)
    if IMG_CH == 1:
        plt.imshow(noisy_image[0][0].cpu(), cmap='gray')
    else:
        img = noisy_image[0].permute(1, 2, 0).cpu()
        if img.min() < 0:
            img = (img + 1) / 2
        plt.imshow(img)
    plt.title(f'{int((i/num_steps) * 100)}% Noise')
    plt.axis('off')
plt.show()

# Show an example of noise progression on a real image
sample_batch = next(iter(train_loader)) # Get first batch
sample_image = sample_batch[0][0].to(device) # Get first image
show_noise_progression(sample_image)

# Student Activity: Try different noise schedules
# Uncomment and modify these lines to experiment:
"""
# Try a non-linear noise schedule
beta_alt = torch.linspace(beta_start, beta_end, n_steps)**2
alpha_alt = 1 - beta_alt
alpha_bar_alt = torch.cumprod(alpha_alt, dim=0)
# How would this affect the diffusion process?
"""

```




```
[ ]: '\n# Try a non-linear noise schedule\nbeta_alt = torch.linspace(beta_start,
beta_end, n_steps)**2\nalpha_alt = 1 - beta_alt\nalpha_bar_alt =
torch.cumprod(alpha_alt, dim=0)\n# How would this affect the diffusion
process?\n'
```

1.6 Step 5: Training Our Model

Now we'll teach our AI to generate images. This process: 1. Takes a clear image 2. Adds random noise to it 3. Asks our AI to predict what noise was added 4. Helps our AI learn from its mistakes

This will take a while, but we'll see progress as it learns!

```
[ ]: # # Create our model and move it to GPU if available
# model = UNet(
#     T=n_steps,
#     img_ch=IMG_CH,
#     img_size=IMG_SIZE,
#     down_chs=(32, 64, 128),
#     t_embed_dim=8,
#     c_embed_dim=N_CLASSES
# ).to(device)

# RECREATE the model with the fixed UNet architecture
model = UNet(
    T=n_steps,                # Number of diffusion time steps
    img_ch=IMG_CH,           # Number of channels in our images (1 for
    ↪ grayscale, 3 for RGB)
    img_size=IMG_SIZE,       # Size of input images (28 for MNIST, 32 for
    ↪ CIFAR-10)
    down_chs=(32, 64, 128), # Channel dimensions for each downsampling level
    t_embed_dim=128,         # Dimension for time step embeddings
    c_embed_dim=N_CLASSES    # Number of classes for conditioning
).to(device)

print(" Model recreated successfully with fixed architecture!")

# Print model summary
print(f"\n{'='*50}")
print("MODEL ARCHITECTURE SUMMARY")
print(f"{'='*50}")
print(f"Input resolution: {IMG_SIZE}x{IMG_SIZE}")
print(f"Input channels: {IMG_CH}")
print(f"Time steps: {n_steps}")
print(f"Condition classes: {N_CLASSES}")
print(f"GPU acceleration: {'Yes' if device.type == 'cuda' else 'No'})
```

```

# Validate model parameters and estimate memory requirements
def count_parameters(model):
    """Count total and trainable parameters in the model."""
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.
        ↳requires_grad)
    return total_params, trainable_params

def estimate_memory(model, batch_size=BATCH_SIZE, img_size=IMG_SIZE,
    ↳img_ch=IMG_CH):
    """Estimate GPU memory usage for the model."""
    # Model parameters memory
    param_memory = sum(p.numel() * p.element_size() for p in model.parameters())

    # Activations memory (rough estimate for one batch)
    activation_memory = batch_size * img_ch * img_size * img_size * 4 # 4
    ↳bytes per float32

    # Total in MB
    total_mb = (param_memory + activation_memory) / (1024 ** 2)
    return total_mb

total_params, trainable_params = count_parameters(model)
memory_mb = estimate_memory(model)

print(f"\nModel Parameters:")
print(f"  Total parameters: {total_params:,}")
print(f"  Trainable parameters: {trainable_params:,}")
print(f"  Estimated GPU memory: {memory_mb:.2f} MB")

# Verify data ranges and integrity
def check_data_ranges(dataloader, name="Dataset"):
    """Check min/max values and types in a dataloader."""
    sample_batch = next(iter(dataloader))
    images, labels = sample_batch

    print(f"\n{name} Statistics:")
    print(f"  Image shape: {images.shape}")
    print(f"  Image dtype: {images.dtype}")
    print(f"  Value range: [{images.min():.3f}, {images.max():.3f}]")
    print(f"  Label range: [{labels.min():}, {labels.max():}]")
    print(f"  Batch size: {len(images)}")

check_data_ranges(train_loader, "Training Data")
check_data_ranges(val_loader, "Validation Data")

```

```

# Set up the optimizer with parameters tuned for diffusion models
# Note: Lower learning rates tend to work better for diffusion models
initial_lr = 0.001 # Starting learning rate
weight_decay = 1e-5 # L2 regularization to prevent overfitting

optimizer = Adam(
    model.parameters(),
    lr=initial_lr,
    weight_decay=weight_decay
)

# Learning rate scheduler to reduce LR when validation loss plateaus
# This helps fine-tune the model toward the end of training
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',           # Reduce LR when monitored value stops decreasing
    factor=0.5,           # Multiply LR by this factor
    patience=5,           # Number of epochs with no improvement after which
    ↪LR will be reduced
    # verbose=True,       # Print message when LR is reduced
    min_lr=1e-6           # Lower bound on the learning rate
)

print("\n" + "="*50)
print("OPTIMIZER AND SCHEDULER CONFIGURED")
print("="*50)
print(f"Initial learning rate: {initial_lr}")
print(f"Weight decay: {weight_decay}")
print(f"Scheduler patience: 5 epochs")
print(f"Minimum learning rate: 1e-6")

# STUDENT EXPERIMENT:
# Try different channel configurations and see how they affect:
# 1. Model size (parameter count)
# 2. Training time
# 3. Generated image quality
#
# Suggestions:
# - Smaller: down_chs=(16, 32, 64)
# - Larger: down_chs=(64, 128, 256, 512)
# Add this to verify
print("UNet created successfully")
for name, param in model.named_parameters():
    if 'upsample' in name and 'weight' in name:
        print(f"{name}: {param.shape}")

```

Created DownBlock: in_chs=32, out_chs=64, spatial_reduction=2x

```
Created DownBlock: in_chs=64, out_chs=128, spatial_reduction=2x
Created UNet with 3 scale levels
Channel dimensions: (32, 64, 128)
Model recreated successfully with fixed architecture!
```

```
=====
MODEL ARCHITECTURE SUMMARY
=====
```

```
Input resolution: 28x28
Input channels: 1
Time steps: 100
Condition classes: 10
GPU acceleration: Yes
```

```
Model Parameters:
```

```
Total parameters: 1,542,401
Trainable parameters: 1,542,401
Estimated GPU memory: 5.98 MB
```

```
Training Data Statistics:
```

```
Image shape: torch.Size([32, 1, 28, 28])
Image dtype: torch.float32
Value range: [-1.000, 1.000]
Label range: [0, 9]
Batch size: 32
```

```
Validation Data Statistics:
```

```
Image shape: torch.Size([32, 1, 28, 28])
Image dtype: torch.float32
Value range: [-1.000, 1.000]
Label range: [0, 9]
Batch size: 32
```

```
=====
OPTIMIZER AND SCHEDULER CONFIGURED
=====
```

```
Initial learning rate: 0.001
Weight decay: 1e-05
Scheduler patience: 5 epochs
Minimum learning rate: 1e-6
UNet created successfully
ups.0.upsample.weight: torch.Size([128, 64, 2, 2])
ups.1.upsample.weight: torch.Size([64, 32, 2, 2])
```

```
[ ]: import os
      from torchvision.utils import make_grid
```

```
# This contains: validate_model_parameters, verify_data_range,  
# generate_samples, safe_save_model from your documents
```

```
[ ]: # Define helper functions needed for training and evaluation  
def validate_model_parameters(model):  
    """  
    Counts model parameters and estimates memory usage.  
    """  
    total_params = sum(p.numel() for p in model.parameters())  
    trainable_params = sum(p.numel() for p in model.parameters() if p.  
↪requires_grad)  
  
    print(f"Total parameters: {total_params:,}")  
    print(f"Trainable parameters: {trainable_params:,}")  
  
    # Estimate memory requirements (very approximate)  
    param_memory = total_params * 4 / (1024 ** 2) # MB for params (float32)  
    grad_memory = trainable_params * 4 / (1024 ** 2) # MB for gradients  
    buffer_memory = param_memory * 2 # Optimizer state, forward activations, ↵  
↪etc.  
  
    print(f"Estimated GPU memory usage: {param_memory + grad_memory + ↵  
↪buffer_memory:.1f} MB")  
  
# Define helper functions for verifying data ranges  
def verify_data_range(dataloader, name="Dataset"):  
    """  
    Verifies the range and integrity of the data.  
    """  
    batch = next(iter(dataloader))[0]  
    print(f"\n{name} range check:")  
    print(f"Shape: {batch.shape}")  
    print(f>Data type: {batch.dtype}")  
    print(f"Min value: {batch.min().item():.2f}")  
    print(f"Max value: {batch.max().item():.2f}")  
    print(f"Contains NaN: {torch.isnan(batch).any().item()}")  
    print(f"Contains Inf: {torch.isinf(batch).any().item()}")  
  
# Define helper functions for generating samples during training  
def generate_samples(model, n_samples=10):  
    """  
    Generates sample images using the model for visualization during training.  
    """  
    model.eval()  
    with torch.no_grad():  
        # Generate digits 0-9 for visualization  
        samples = []
```

```

for digit in range(min(n_samples, 10)):
    # Start with random noise
    x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

    # Set up conditioning for the digit
    c = torch.tensor([digit]).to(device)
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Remove noise step by step
    for t in range(n_steps-1, -1, -1):
        t_batch = torch.full((1,), t).to(device)
        x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

    samples.append(x)

# Combine samples and display
samples = torch.cat(samples, dim=0)
grid = make_grid(samples, nrow=min(n_samples, 5), normalize=True)

plt.figure(figsize=(10, 4))

# Display based on channel configuration
if IMG_CH == 1:
    plt.imshow(grid[0].cpu(), cmap='gray')
else:
    plt.imshow(grid.permute(1, 2, 0).cpu())

plt.axis('off')
plt.title('Generated Samples')
plt.show()

# Define helper functions for safely saving models
def safe_save_model(model, path, optimizer=None, epoch=None, best_loss=None):
    """
    Safely saves model with error handling and backup.
    """
    try:
        # Create a dictionary with all the elements to save
        save_dict = {
            'model_state_dict': model.state_dict(),
        }

        # Add optional elements if provided
        if optimizer is not None:
            save_dict['optimizer_state_dict'] = optimizer.state_dict()
        if epoch is not None:

```

```

        save_dict['epoch'] = epoch
    if best_loss is not None:
        save_dict['best_loss'] = best_loss

    # Create a backup of previous checkpoint if it exists
    if os.path.exists(path):
        backup_path = path + '.backup'
        try:
            os.replace(path, backup_path)
            print(f"Created backup at {backup_path}")
        except Exception as e:
            print(f"Warning: Could not create backup - {e}")

    # Save the new checkpoint
    torch.save(save_dict, path)
    print(f"Model successfully saved to {path}")

except Exception as e:
    print(f"Error saving model: {e}")
    print("Attempting emergency save...")

    try:
        emergency_path = path + '.emergency'
        torch.save(model.state_dict(), emergency_path)
        print(f"Emergency save successful: {emergency_path}")
    except:
        print("Emergency save failed. Could not save model.")

```

```

[ ]: # Implementation of the training step function
def train_step(x, c):
    """
    Performs a single training step for the diffusion model.

    This function:
    1. Prepares class conditioning
    2. Samples random timesteps for each image
    3. Adds corresponding noise to the images
    4. Asks the model to predict the noise
    5. Calculates the loss between predicted and actual noise

    Args:
        x (torch.Tensor): Batch of clean images [batch_size, channels, height, width]
        c (torch.Tensor): Batch of class labels [batch_size]

    Returns:
        torch.Tensor: Mean squared error loss value
    """

```

```

"""
# Convert number labels to one-hot encoding for class conditioning
# Example: Label 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0] for MNIST
c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

# Create conditioning mask (all ones for standard training)
# This would be used for classifier-free guidance if implemented
c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

# Pick random timesteps for each image in the batch
# Different timesteps allow the model to learn the entire diffusion process
t = torch.randint(0, n_steps, (x.shape[0],)).to(device)

# Add noise to images according to the forward diffusion process
# This simulates images at different stages of the diffusion process
x_t, noise = add_noise(x, t)

# The model tries to predict the exact noise that was added
# This is the core learning objective of diffusion models
predicted_noise = model(x_t, t, c_one_hot, c_mask)

# Calculate loss: how accurately did the model predict the noise?
# MSE loss works well for image-based diffusion models
loss = F.mse_loss(predicted_noise, noise)

return loss

```

```

[ ]: import torch.nn.functional as F

def forward(self, x, skip):
    # Upsample x first
    x = self.upsample(x)

    # Align spatial dims to the skip connection's size
    if x.shape != skip.shape:
        x = F.interpolate(x, size=skip.shape[2:], mode='bilinear',
↪align_corners=False)

    # Now safe to concatenate
    x = torch.cat([x, skip], dim=1)
    return self.conv(x)

```

```

[ ]: import traceback

try:
    for step, (images, labels) in enumerate(train_loader):
        images = images.to(device)

```



```

        labels = labels.to(device)
        loss = train_step(images, labels)
        break # just test one batch
except Exception as e:
    traceback.print_exc()

```

```

[ ]: # Test embedding output shapes
dummy_t = torch.zeros(4, dtype=torch.long).to(device)
dummy_c = torch.zeros(4, N_CLASSES).to(device)
dummy_mask = torch.ones(4, 1).to(device)

t_emb = model.time_embed(dummy_t)
c_emb = model.class_embed(dummy_c)
print(f"t_emb shape: {t_emb.shape}") # should be [4, 128]
print(f"c_emb shape: {c_emb.shape}") # should be [4, 128]

```

```

t_emb shape: torch.Size([4, 128])
c_emb shape: torch.Size([4, 128, 1, 1])

```

```

[ ]: for i, up in enumerate(model.ups):
    print(f"\nups.{i} parameters:")
    for name, param in up.named_parameters():
        print(f"    {name}: {param.shape}")

```

```

ups.0 parameters:
upsample.weight: torch.Size([128, 64, 2, 2])
upsample.bias: torch.Size([64])
conv1.model.0.weight: torch.Size([64, 128, 3, 3])
conv1.model.0.bias: torch.Size([64])
conv1.model.1.weight: torch.Size([64])
conv1.model.1.bias: torch.Size([64])
conv2.model.0.weight: torch.Size([64, 64, 3, 3])
conv2.model.0.bias: torch.Size([64])
conv2.model.1.weight: torch.Size([64])
conv2.model.1.bias: torch.Size([64])

```

```

ups.1 parameters:
upsample.weight: torch.Size([64, 32, 2, 2])
upsample.bias: torch.Size([32])
conv1.model.0.weight: torch.Size([32, 64, 3, 3])
conv1.model.0.bias: torch.Size([32])
conv1.model.1.weight: torch.Size([32])
conv1.model.1.bias: torch.Size([32])
conv2.model.0.weight: torch.Size([32, 32, 3, 3])
conv2.model.0.bias: torch.Size([32])
conv2.model.1.weight: torch.Size([32])

```

```
conv2.model.1.bias: torch.Size([32])
```

```
[ ]: print(model.final_conv.weight.shape) # should be [img_ch, down_chs[0], 1, 1]
for i, down in enumerate(model.downs):
    for name, param in down.named_parameters():
        if 'weight' in name:
            print(f"downs.{i} {name}: {param.shape}")
            break
```

```
torch.Size([1, 32, 1, 1])
```

```
downs.0 model.0.model.0.weight: torch.Size([64, 32, 3, 3])
```

```
downs.1 model.0.model.0.weight: torch.Size([128, 64, 3, 3])
```

```
[ ]: # Check the instantiation cell is using correct down_chs
print("Number of down blocks:", len(model.downs))
print("Number of up blocks:", len(model.ups))
```

```
Number of down blocks: 2
```

```
Number of up blocks: 2
```

```
[ ]: # Find which kernel file has UpBlock
import ipynbname
print(UpBlock.__module__)
print(UpBlock)
```

```
__main__
<class '__main__.UpBlock'>
```

```
[ ]: import dis
dis.dis(UpBlock.__init__)
```

	0 COPY_FREE_VARS	1
16	2 RESUME	0
17	4 LOAD_GLOBAL	0 (super)
	14 LOAD_DEREF	4 (__class__)
	16 LOAD_FAST	0 (self)
	18 LOAD_SUPER_ATTR	5 (NULL self + __init__)
	22 CALL	0
	30 POP_TOP	
20	32 LOAD_GLOBAL	4 (nn)
	42 LOAD_ATTR	7 (NULL self + ConvTranspose2d)
21	62 LOAD_FAST	1 (in_chs)
22	64 LOAD_FAST	2 (out_chs)

23	66 LOAD_CONST	1 (2)
24	68 LOAD_CONST	1 (2)
20	70 KW_NAMES	2 (('in_channels', 'out_channels',
	'kernel_size', 'stride'))	
	72 CALL	4
	80 LOAD_FAST	0 (self)
	82 STORE_ATTR	4 (upsample)
29	92 LOAD_GLOBAL	11 (NULL + GELUConvBlock)
	102 LOAD_FAST	2 (out_chs)
	104 LOAD_FAST	2 (out_chs)
	106 BINARY_OP	0 (+)
	110 LOAD_FAST	2 (out_chs)
	112 LOAD_FAST	3 (group_size)
	114 KW_NAMES	3 (('group_size',))
	116 CALL	3
	124 LOAD_FAST	0 (self)
	126 STORE_ATTR	6 (conv1)
30	136 LOAD_GLOBAL	11 (NULL + GELUConvBlock)
	146 LOAD_FAST	2 (out_chs)
	148 LOAD_FAST	2 (out_chs)
	150 LOAD_FAST	3 (group_size)
	152 KW_NAMES	3 (('group_size',))
	154 CALL	3
	162 LOAD_FAST	0 (self)
	164 STORE_ATTR	7 (conv2)
	174 RETURN_CONST	0 (None)

```
[ ]: # Implementation of the main training loop
# Training configuration
early_stopping_patience = 10 # Number of epochs without improvement before
    ↪stopping
gradient_clip_value = 1.0 # Maximum gradient norm for stability
display_frequency = 100 # How often to show progress (in steps)
generate_frequency = 500 # How often to generate samples (in steps)

# Progress tracking variables
best_loss = float('inf')
train_losses = []
val_losses = []
no_improve_epochs = 0

# Training loop
```

```

print("\n" + "="*50)
print("STARTING TRAINING")
print("="*50)
model.train()

# Wrap the training loop in a try-except block for better error handling:
# Your code for the training loop
# Hint: Use a try-except block for better error handling
# Process each epoch and each batch, with validation after each epoch

# Enter your code here:

#try:
for epoch in range(EPOCHS):
    print(f"\nEpoch {epoch+1}/{EPOCHS}")
    print("-" * 20)

    # Training phase
    model.train()
    epoch_losses = []

    # Process each batch
    for step, (images, labels) in enumerate(train_loader): # Fixed:
        ↪dataloader → train_data_loader
        images = images.to(device)
        labels = labels.to(device)

        # Training step
        optimizer.zero_grad()
        loss = train_step(images, labels)
        loss.backward()

        # Add gradient clipping for stability
        torch.nn.utils.clip_grad_norm_(model.parameters(),
        ↪max_norm=gradient_clip_value)

        optimizer.step()
        epoch_losses.append(loss.item())

    # Show progress at regular intervals
    if step % display_frequency == 0:
        print(f" Step {step}/{len(train_loader)}, Loss: {loss.item():.
        ↪4f}")

    # Generate samples less frequently to save time
    if step % generate_frequency == 0 and step > 0:
        print(" Generating samples...")

```

```

        generate_samples(model, n_samples=5)

    # End of epoch - calculate average training loss
    avg_train_loss = sum(epoch_losses) / len(epoch_losses)
    train_losses.append(avg_train_loss)
    print(f"\nTraining - Epoch {epoch+1} average loss: {avg_train_loss:.
↪4f}")

    # Validation phase
    model.eval()
    val_epoch_losses = []
    print("Running validation...")

    with torch.no_grad(): # Disable gradients for validation
        for val_images, val_labels in val_loader:
            val_images = val_images.to(device)
            val_labels = val_labels.to(device)

            # Calculate validation loss
            val_loss = train_step(val_images, val_labels)
            val_epoch_losses.append(val_loss.item())

    # Calculate average validation loss
    avg_val_loss = sum(val_epoch_losses) / len(val_epoch_losses)
    val_losses.append(avg_val_loss)
    print(f"Validation - Epoch {epoch+1} average loss: {avg_val_loss:.4f}")

    # Learning rate scheduling based on validation loss
    scheduler.step(avg_val_loss)
    current_lr = optimizer.param_groups[0]['lr']
    print(f"Learning rate: {current_lr:.6f}")

    # Generate samples at the end of each epoch
    if epoch % 2 == 0 or epoch == EPOCHS - 1:
        print("\nGenerating samples for visual progress check...")
        generate_samples(model, n_samples=10)

    # Save best model based on validation loss
    if avg_val_loss < best_loss:
        best_loss = avg_val_loss
        # Use safe_save_model instead of just saving state_dict
        safe_save_model(model, 'best_diffusion_model.pt', optimizer, epoch,
↪best_loss)
        print(f" New best model saved! (Val Loss: {best_loss:.4f})")
        no_improve_epochs = 0
    else:
        no_improve_epochs += 1

```

```

        print(f"No improvement for {no_improve_epochs}/
↳{early_stopping_patience} epochs")

    # Early stopping
    if no_improve_epochs >= early_stopping_patience:
        print("\nEarly stopping triggered! No improvement in validation_
↳loss.")
        break

    # Plot loss curves every few epochs
    if epoch % 5 == 0 or epoch == EPOCHS - 1:
        plt.figure(figsize=(10, 5))
        plt.plot(train_losses, label='Training Loss')
        plt.plot(val_losses, label='Validation Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Training and Validation Loss')
        plt.legend()
        plt.grid(True)
        plt.show()

# Final wrap-up
print("\n" + "="*50)
print("TRAINING COMPLETE")
print("="*50)
print(f"Best validation loss: {best_loss:.4f}")

# Generate final samples
print("Generating final samples...")
generate_samples(model, n_samples=10)

# Display final loss curves
plt.figure(figsize=(12, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

# Clean up memory

```

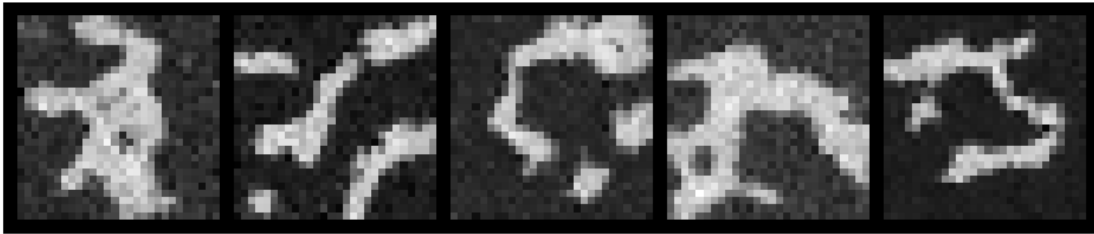
```
torch.cuda.empty_cache()
```

```
=====
STARTING TRAINING
=====
```

Epoch 1/30

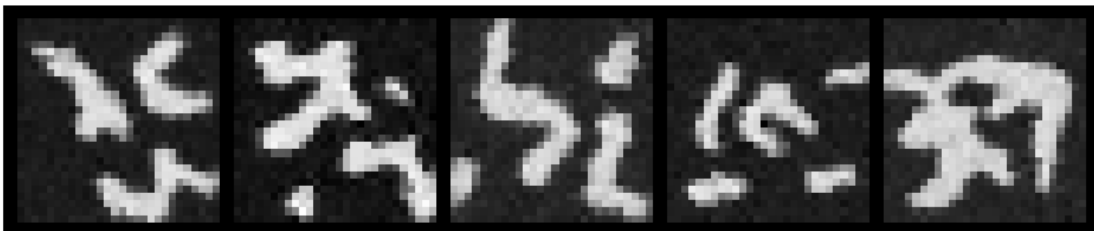
```
-----
Step 0/1500, Loss: 1.1873
Step 100/1500, Loss: 0.1410
Step 200/1500, Loss: 0.1242
Step 300/1500, Loss: 0.0987
Step 400/1500, Loss: 0.1003
Step 500/1500, Loss: 0.0784
Generating samples...
```

Generated Samples



```
Step 600/1500, Loss: 0.0591
Step 700/1500, Loss: 0.0605
Step 800/1500, Loss: 0.0927
Step 900/1500, Loss: 0.0703
Step 1000/1500, Loss: 0.0645
Generating samples...
```

Generated Samples

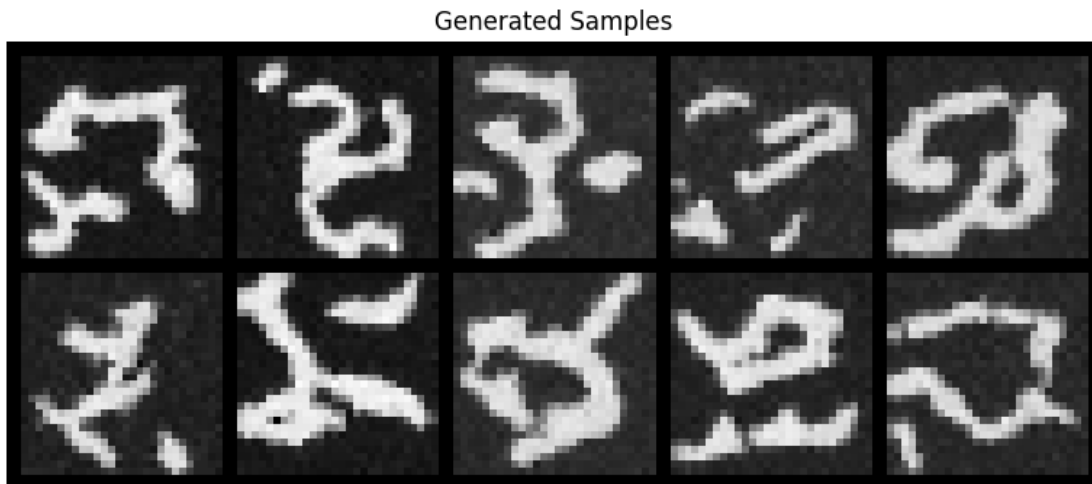


```
Step 1100/1500, Loss: 0.0618
Step 1200/1500, Loss: 0.0684
```

Step 1300/1500, Loss: 0.0716
Step 1400/1500, Loss: 0.0798

Training - Epoch 1 average loss: 0.0915
Running validation...
Validation - Epoch 1 average loss: 0.0712
Learning rate: 0.001000

Generating samples for visual progress check...



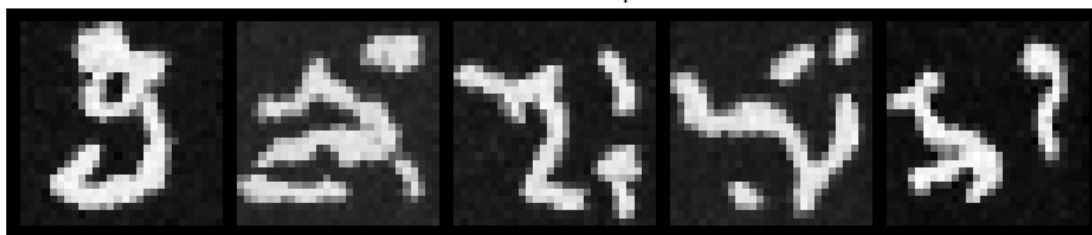
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0712)



Epoch 2/30

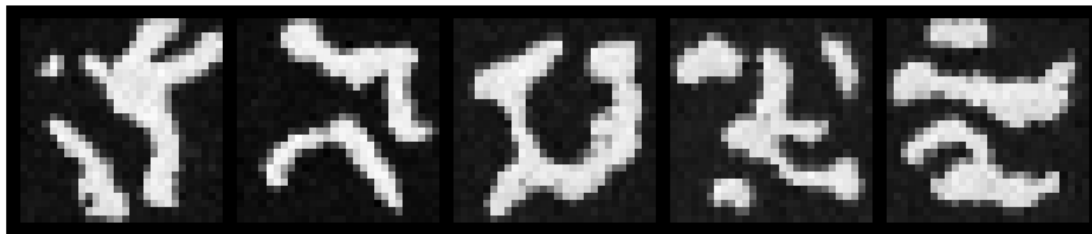
Step 0/1500, Loss: 0.0640
Step 100/1500, Loss: 0.0755
Step 200/1500, Loss: 0.0603
Step 300/1500, Loss: 0.0858
Step 400/1500, Loss: 0.0664
Step 500/1500, Loss: 0.0650
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0820
Step 700/1500, Loss: 0.0697
Step 800/1500, Loss: 0.0670
Step 900/1500, Loss: 0.0803
Step 1000/1500, Loss: 0.0594
Generating samples...

Generated Samples



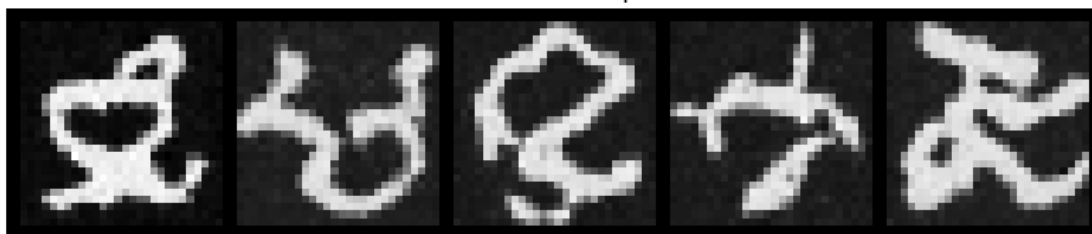
Step 1100/1500, Loss: 0.0726
Step 1200/1500, Loss: 0.0654
Step 1300/1500, Loss: 0.0756
Step 1400/1500, Loss: 0.0577

Training - Epoch 2 average loss: 0.0677
Running validation...
Validation - Epoch 2 average loss: 0.0677
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0677)

Epoch 3/30

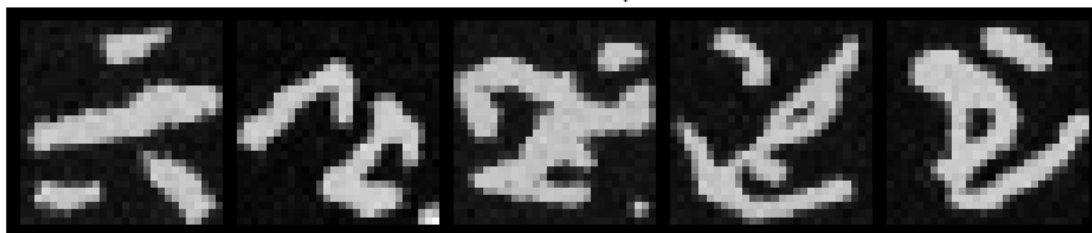
Step 0/1500, Loss: 0.0696
Step 100/1500, Loss: 0.0687
Step 200/1500, Loss: 0.0585
Step 300/1500, Loss: 0.0610
Step 400/1500, Loss: 0.0586
Step 500/1500, Loss: 0.0621
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0694
Step 700/1500, Loss: 0.0537
Step 800/1500, Loss: 0.0684
Step 900/1500, Loss: 0.0614
Step 1000/1500, Loss: 0.0789
Generating samples...

Generated Samples

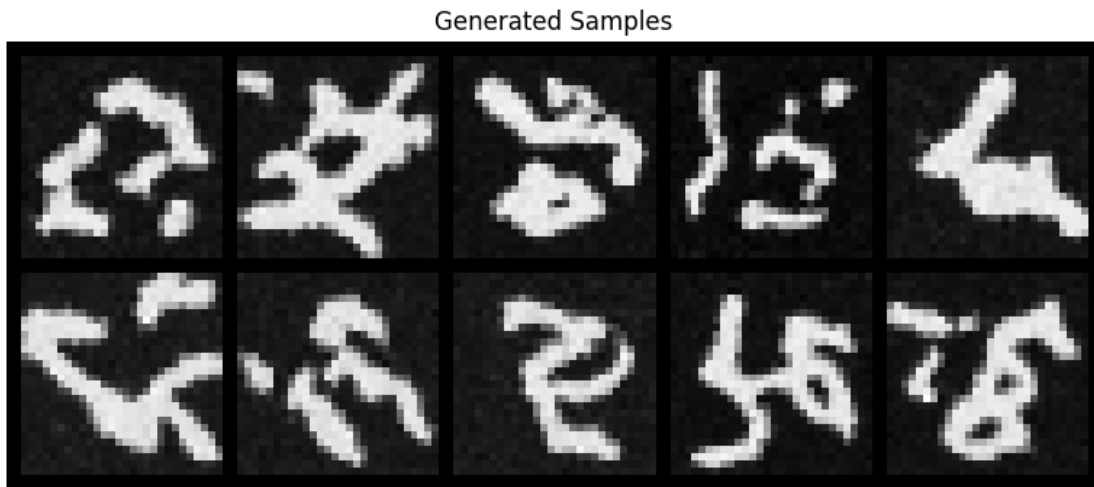


Step 1100/1500, Loss: 0.0713

Step 1200/1500, Loss: 0.0619
Step 1300/1500, Loss: 0.0613
Step 1400/1500, Loss: 0.0739

Training - Epoch 3 average loss: 0.0645
Running validation...
Validation - Epoch 3 average loss: 0.0637
Learning rate: 0.001000

Generating samples for visual progress check...



Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0637)

Epoch 4/30

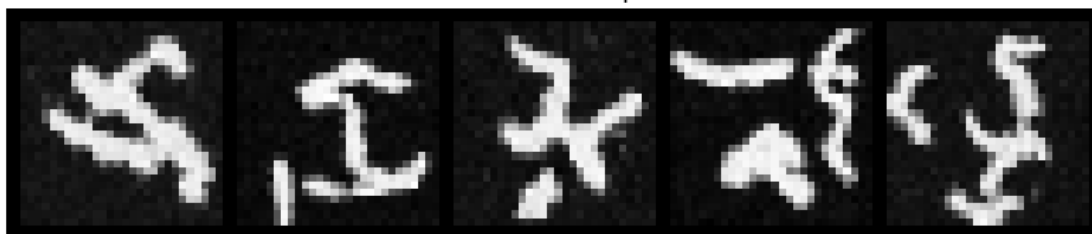
Step 0/1500, Loss: 0.0649
Step 100/1500, Loss: 0.0670
Step 200/1500, Loss: 0.0639
Step 300/1500, Loss: 0.0755
Step 400/1500, Loss: 0.0579
Step 500/1500, Loss: 0.0636
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0501
Step 700/1500, Loss: 0.0552
Step 800/1500, Loss: 0.0665
Step 900/1500, Loss: 0.0710
Step 1000/1500, Loss: 0.0520
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0605
Step 1200/1500, Loss: 0.0503
Step 1300/1500, Loss: 0.0718
Step 1400/1500, Loss: 0.0542

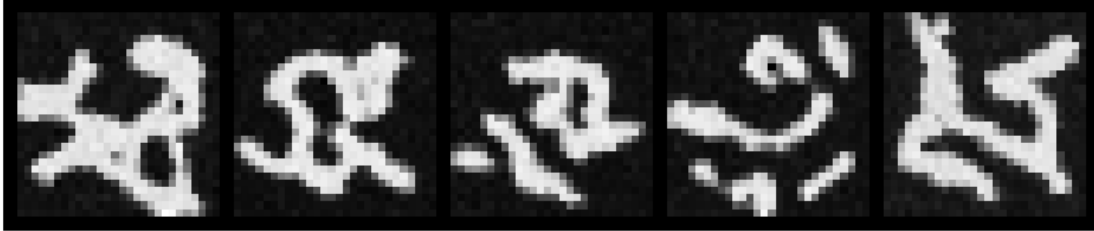
Training - Epoch 4 average loss: 0.0628
Running validation...
Validation - Epoch 4 average loss: 0.0625
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0625)

Epoch 5/30

Step 0/1500, Loss: 0.0545
Step 100/1500, Loss: 0.0695
Step 200/1500, Loss: 0.0585

Step 300/1500, Loss: 0.0642
Step 400/1500, Loss: 0.0700
Step 500/1500, Loss: 0.0592
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0664
Step 700/1500, Loss: 0.0579
Step 800/1500, Loss: 0.0595
Step 900/1500, Loss: 0.0658
Step 1000/1500, Loss: 0.0821
Generating samples...

Generated Samples

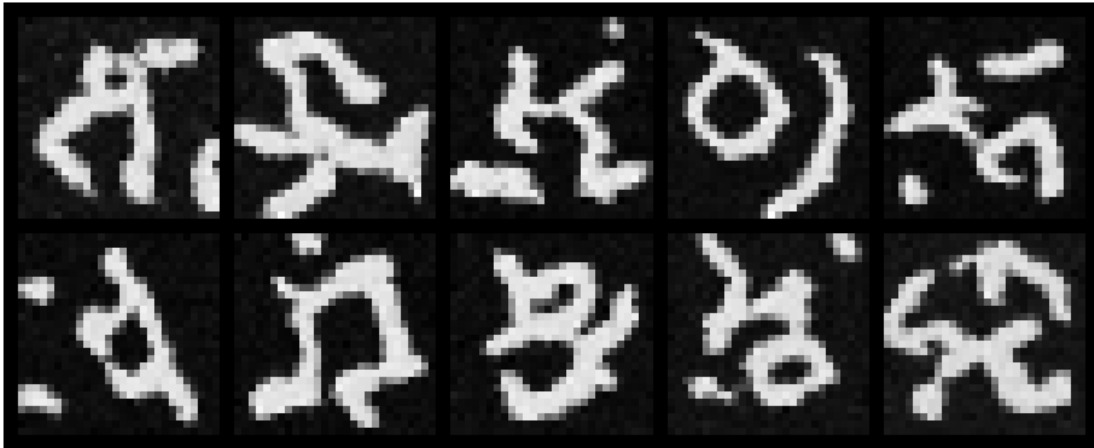


Step 1100/1500, Loss: 0.0719
Step 1200/1500, Loss: 0.0540
Step 1300/1500, Loss: 0.0601
Step 1400/1500, Loss: 0.0544

Training - Epoch 5 average loss: 0.0623
Running validation...
Validation - Epoch 5 average loss: 0.0629
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

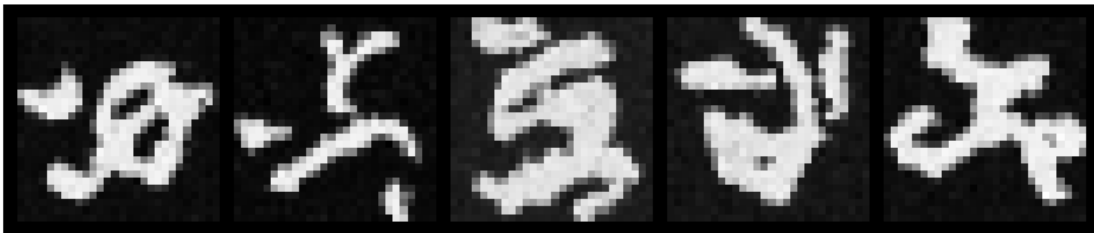


No improvement for 1/10 epochs

Epoch 6/30

Step 0/1500, Loss: 0.0687
Step 100/1500, Loss: 0.0517
Step 200/1500, Loss: 0.0597
Step 300/1500, Loss: 0.0718
Step 400/1500, Loss: 0.0670
Step 500/1500, Loss: 0.0527
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0577
Step 700/1500, Loss: 0.0435
Step 800/1500, Loss: 0.0473
Step 900/1500, Loss: 0.0576
Step 1000/1500, Loss: 0.0623
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0767
Step 1200/1500, Loss: 0.0578
Step 1300/1500, Loss: 0.0572
Step 1400/1500, Loss: 0.0622

Training - Epoch 6 average loss: 0.0610

Running validation...

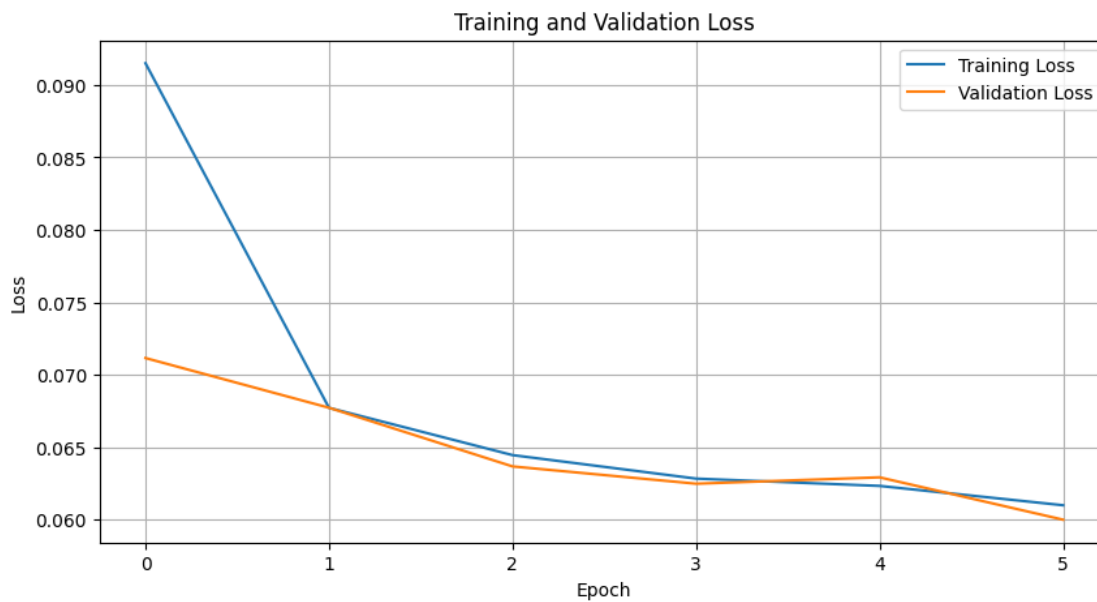
Validation - Epoch 6 average loss: 0.0600

Learning rate: 0.001000

Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt

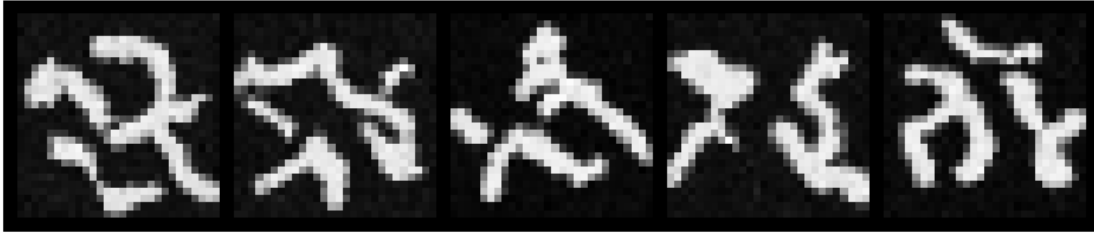
New best model saved! (Val Loss: 0.0600)



Epoch 7/30

Step 0/1500, Loss: 0.0647
Step 100/1500, Loss: 0.0573
Step 200/1500, Loss: 0.0627
Step 300/1500, Loss: 0.0488
Step 400/1500, Loss: 0.0559
Step 500/1500, Loss: 0.0504
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0607
Step 700/1500, Loss: 0.0737
Step 800/1500, Loss: 0.0594
Step 900/1500, Loss: 0.0612
Step 1000/1500, Loss: 0.0634
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0525
Step 1200/1500, Loss: 0.0644
Step 1300/1500, Loss: 0.0671
Step 1400/1500, Loss: 0.0530

Training - Epoch 7 average loss: 0.0602
Running validation...
Validation - Epoch 7 average loss: 0.0600
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0600)

Epoch 8/30

Step 0/1500, Loss: 0.0579
Step 100/1500, Loss: 0.0602
Step 200/1500, Loss: 0.0593
Step 300/1500, Loss: 0.0541
Step 400/1500, Loss: 0.0486
Step 500/1500, Loss: 0.0676
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0506
Step 700/1500, Loss: 0.0480
Step 800/1500, Loss: 0.0651
Step 900/1500, Loss: 0.0569
Step 1000/1500, Loss: 0.0668

Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0730

Step 1200/1500, Loss: 0.0562

Step 1300/1500, Loss: 0.0659

Step 1400/1500, Loss: 0.0624

Training - Epoch 8 average loss: 0.0598

Running validation...

Validation - Epoch 8 average loss: 0.0598

Learning rate: 0.001000

Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt

New best model saved! (Val Loss: 0.0598)

Epoch 9/30

Step 0/1500, Loss: 0.0588

Step 100/1500, Loss: 0.0612

Step 200/1500, Loss: 0.0507

Step 300/1500, Loss: 0.0585

Step 400/1500, Loss: 0.0534

Step 500/1500, Loss: 0.0597

Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0590

Step 700/1500, Loss: 0.0567
Step 800/1500, Loss: 0.0645
Step 900/1500, Loss: 0.0584
Step 1000/1500, Loss: 0.0492
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0571
Step 1200/1500, Loss: 0.0561
Step 1300/1500, Loss: 0.0590
Step 1400/1500, Loss: 0.0547

Training - Epoch 9 average loss: 0.0594
Running validation...
Validation - Epoch 9 average loss: 0.0581
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



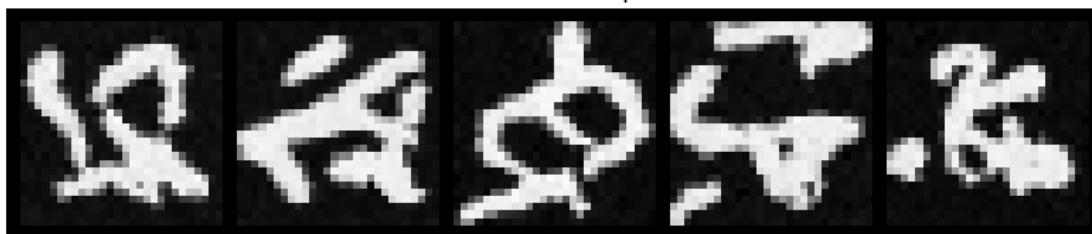
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt

New best model saved! (Val Loss: 0.0581)

Epoch 10/30

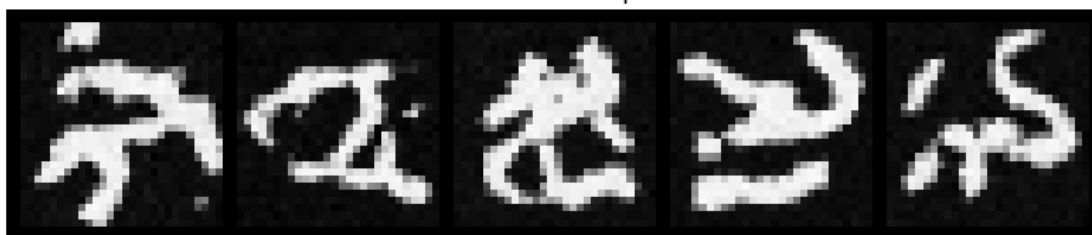
Step 0/1500, Loss: 0.0599
Step 100/1500, Loss: 0.0621
Step 200/1500, Loss: 0.0716
Step 300/1500, Loss: 0.0649
Step 400/1500, Loss: 0.0584
Step 500/1500, Loss: 0.0514
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0664
Step 700/1500, Loss: 0.0601
Step 800/1500, Loss: 0.0595
Step 900/1500, Loss: 0.0505
Step 1000/1500, Loss: 0.0737
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0615
Step 1200/1500, Loss: 0.0550
Step 1300/1500, Loss: 0.0641
Step 1400/1500, Loss: 0.0549

Training - Epoch 10 average loss: 0.0589
Running validation...

Validation - Epoch 10 average loss: 0.0593
Learning rate: 0.001000
No improvement for 1/10 epochs

Epoch 11/30

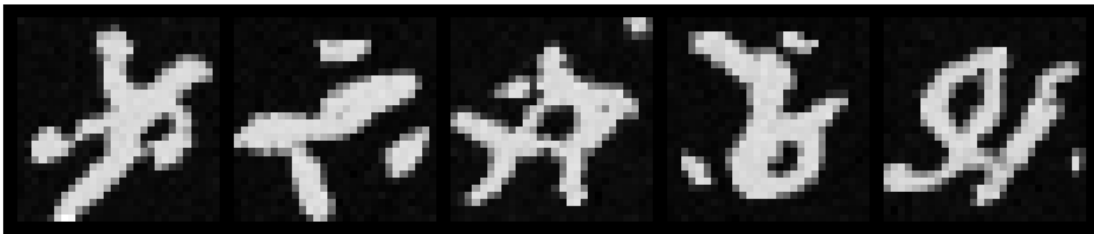
Step 0/1500, Loss: 0.0660
Step 100/1500, Loss: 0.0678
Step 200/1500, Loss: 0.0559
Step 300/1500, Loss: 0.0643
Step 400/1500, Loss: 0.0503
Step 500/1500, Loss: 0.0609
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0621
Step 700/1500, Loss: 0.0537
Step 800/1500, Loss: 0.0423
Step 900/1500, Loss: 0.0596
Step 1000/1500, Loss: 0.0605
Generating samples...

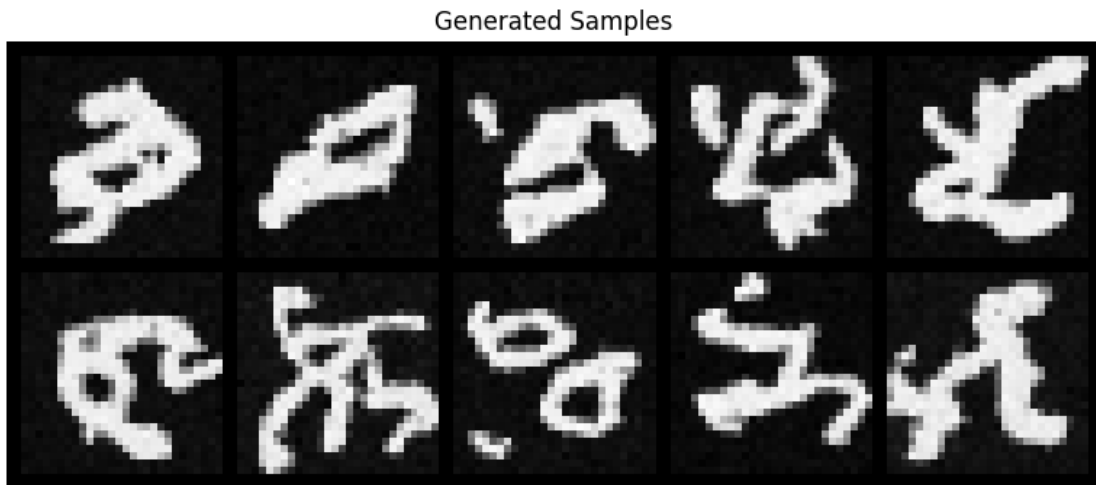
Generated Samples



Step 1100/1500, Loss: 0.0541
Step 1200/1500, Loss: 0.0562
Step 1300/1500, Loss: 0.0591
Step 1400/1500, Loss: 0.0642

Training - Epoch 11 average loss: 0.0587
Running validation...
Validation - Epoch 11 average loss: 0.0594
Learning rate: 0.001000

Generating samples for visual progress check...



No improvement for 2/10 epochs



Epoch 12/30

Step 0/1500, Loss: 0.0659
Step 100/1500, Loss: 0.0619
Step 200/1500, Loss: 0.0561
Step 300/1500, Loss: 0.0561
Step 400/1500, Loss: 0.0581
Step 500/1500, Loss: 0.0581
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0424
Step 700/1500, Loss: 0.0543
Step 800/1500, Loss: 0.0649
Step 900/1500, Loss: 0.0514
Step 1000/1500, Loss: 0.0626
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0516
Step 1200/1500, Loss: 0.0496
Step 1300/1500, Loss: 0.0522
Step 1400/1500, Loss: 0.0503

Training - Epoch 12 average loss: 0.0584
Running validation...
Validation - Epoch 12 average loss: 0.0574
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0574)

Epoch 13/30

Step 0/1500, Loss: 0.0556
Step 100/1500, Loss: 0.0522
Step 200/1500, Loss: 0.0564
Step 300/1500, Loss: 0.0489
Step 400/1500, Loss: 0.0628
Step 500/1500, Loss: 0.0542
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0579
Step 700/1500, Loss: 0.0668
Step 800/1500, Loss: 0.0527
Step 900/1500, Loss: 0.0660
Step 1000/1500, Loss: 0.0648
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0564
Step 1200/1500, Loss: 0.0645
Step 1300/1500, Loss: 0.0686
Step 1400/1500, Loss: 0.0504

Training - Epoch 13 average loss: 0.0583

Running validation...

Validation - Epoch 13 average loss: 0.0578

Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



No improvement for 1/10 epochs

Epoch 14/30

Step 0/1500, Loss: 0.0547

Step 100/1500, Loss: 0.0537

Step 200/1500, Loss: 0.0579

Step 300/1500, Loss: 0.0497

Step 400/1500, Loss: 0.0505

Step 500/1500, Loss: 0.0586

Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0603

Step 700/1500, Loss: 0.0461

Step 800/1500, Loss: 0.0515
Step 900/1500, Loss: 0.0541
Step 1000/1500, Loss: 0.0590
Generating samples...

Generated Samples



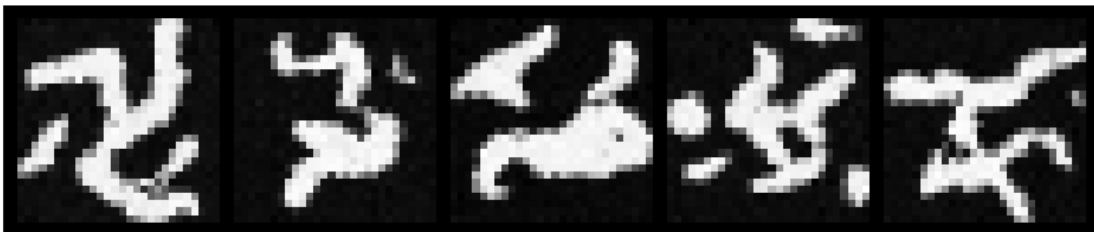
Step 1100/1500, Loss: 0.0595
Step 1200/1500, Loss: 0.0553
Step 1300/1500, Loss: 0.0586
Step 1400/1500, Loss: 0.0442

Training - Epoch 14 average loss: 0.0583
Running validation...
Validation - Epoch 14 average loss: 0.0588
Learning rate: 0.001000
No improvement for 2/10 epochs

Epoch 15/30

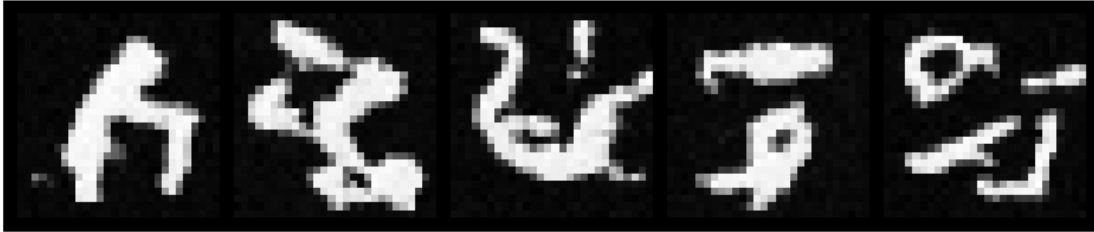
Step 0/1500, Loss: 0.0520
Step 100/1500, Loss: 0.0510
Step 200/1500, Loss: 0.0580
Step 300/1500, Loss: 0.0554
Step 400/1500, Loss: 0.0660
Step 500/1500, Loss: 0.0526
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0517
Step 700/1500, Loss: 0.0522
Step 800/1500, Loss: 0.0729
Step 900/1500, Loss: 0.0545
Step 1000/1500, Loss: 0.0631
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0653
Step 1200/1500, Loss: 0.0514
Step 1300/1500, Loss: 0.0511
Step 1400/1500, Loss: 0.0580

Training - Epoch 15 average loss: 0.0582
Running validation...
Validation - Epoch 15 average loss: 0.0571
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0571)

Epoch 16/30

Step 0/1500, Loss: 0.0533
Step 100/1500, Loss: 0.0608
Step 200/1500, Loss: 0.0550
Step 300/1500, Loss: 0.0530
Step 400/1500, Loss: 0.0643
Step 500/1500, Loss: 0.0613
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0549
Step 700/1500, Loss: 0.0511
Step 800/1500, Loss: 0.0615
Step 900/1500, Loss: 0.0605
Step 1000/1500, Loss: 0.0517
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0708
Step 1200/1500, Loss: 0.0539
Step 1300/1500, Loss: 0.0576
Step 1400/1500, Loss: 0.0564

Training - Epoch 16 average loss: 0.0579

Running validation...

Validation - Epoch 16 average loss: 0.0572

Learning rate: 0.001000

No improvement for 1/10 epochs



Epoch 17/30

Step 0/1500, Loss: 0.0523

Step 100/1500, Loss: 0.0565

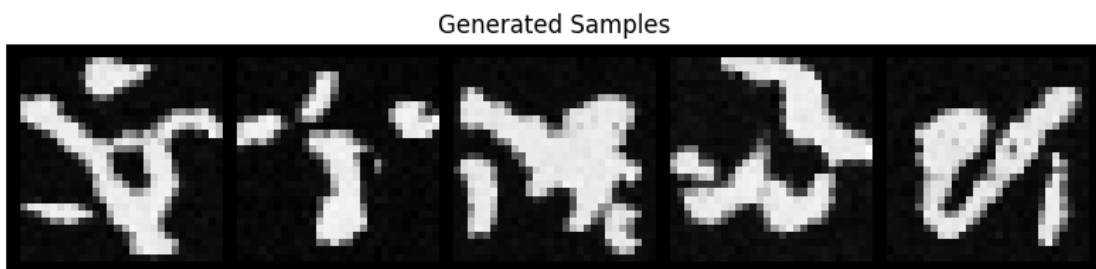
Step 200/1500, Loss: 0.0575

Step 300/1500, Loss: 0.0702

Step 400/1500, Loss: 0.0645

Step 500/1500, Loss: 0.0661

Generating samples...



Step 600/1500, Loss: 0.0637

Step 700/1500, Loss: 0.0606
Step 800/1500, Loss: 0.0544
Step 900/1500, Loss: 0.0537
Step 1000/1500, Loss: 0.0661
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0625
Step 1200/1500, Loss: 0.0582
Step 1300/1500, Loss: 0.0719
Step 1400/1500, Loss: 0.0712

Training - Epoch 17 average loss: 0.0577
Running validation...
Validation - Epoch 17 average loss: 0.0567
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



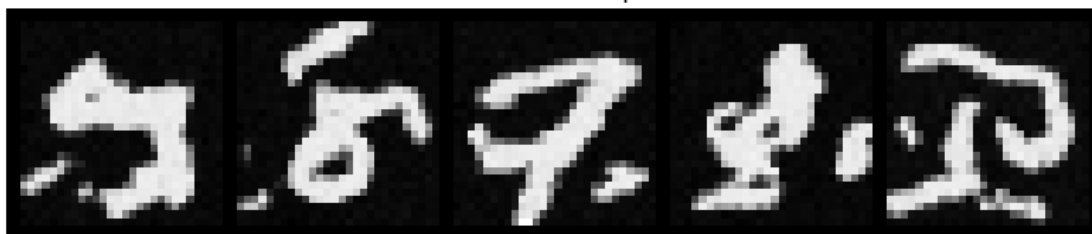
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt

New best model saved! (Val Loss: 0.0567)

Epoch 18/30

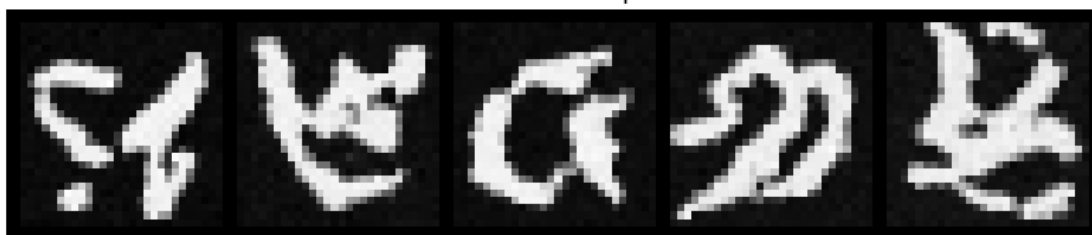
Step 0/1500, Loss: 0.0602
Step 100/1500, Loss: 0.0648
Step 200/1500, Loss: 0.0550
Step 300/1500, Loss: 0.0578
Step 400/1500, Loss: 0.0715
Step 500/1500, Loss: 0.0666
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0690
Step 700/1500, Loss: 0.0587
Step 800/1500, Loss: 0.0654
Step 900/1500, Loss: 0.0555
Step 1000/1500, Loss: 0.0550
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0522
Step 1200/1500, Loss: 0.0501
Step 1300/1500, Loss: 0.0429
Step 1400/1500, Loss: 0.0561

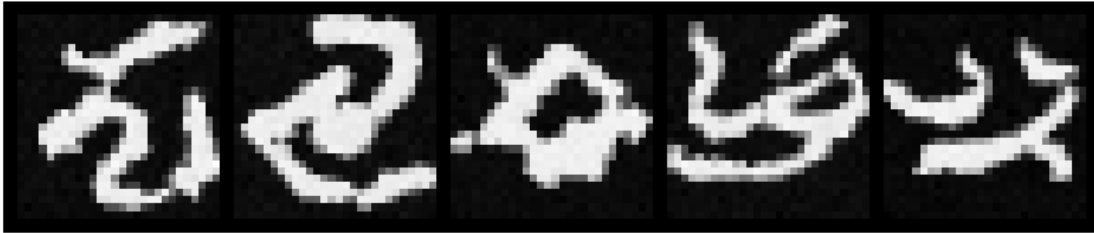
Training - Epoch 18 average loss: 0.0573
Running validation...

Validation - Epoch 18 average loss: 0.0572
Learning rate: 0.001000
No improvement for 1/10 epochs

Epoch 19/30

Step 0/1500, Loss: 0.0609
Step 100/1500, Loss: 0.0532
Step 200/1500, Loss: 0.0515
Step 300/1500, Loss: 0.0543
Step 400/1500, Loss: 0.0432
Step 500/1500, Loss: 0.0617
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0526
Step 700/1500, Loss: 0.0680
Step 800/1500, Loss: 0.0559
Step 900/1500, Loss: 0.0465
Step 1000/1500, Loss: 0.0606
Generating samples...

Generated Samples

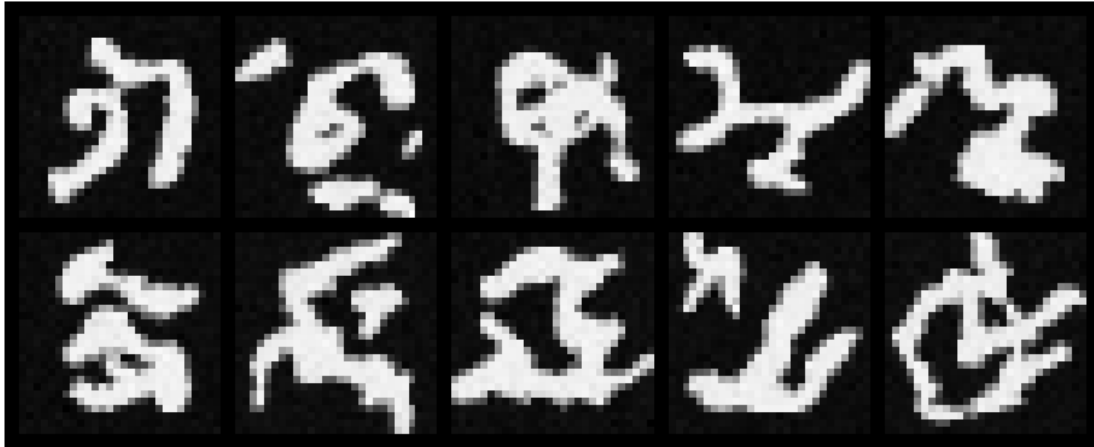


Step 1100/1500, Loss: 0.0461
Step 1200/1500, Loss: 0.0446
Step 1300/1500, Loss: 0.0572
Step 1400/1500, Loss: 0.0527

Training - Epoch 19 average loss: 0.0573
Running validation...
Validation - Epoch 19 average loss: 0.0566
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0566)

Epoch 20/30

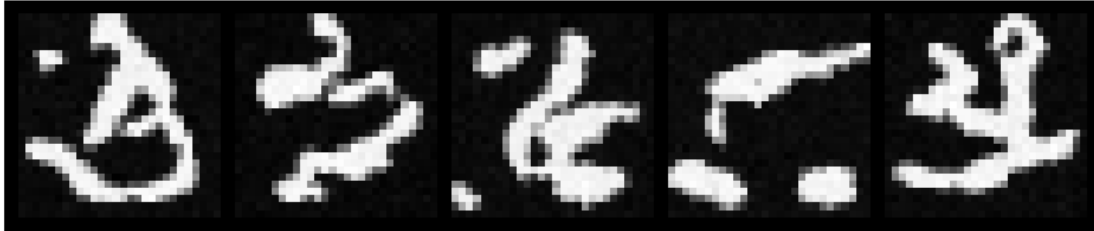
Step 0/1500, Loss: 0.0516
Step 100/1500, Loss: 0.0542
Step 200/1500, Loss: 0.0624
Step 300/1500, Loss: 0.0624
Step 400/1500, Loss: 0.0570
Step 500/1500, Loss: 0.0555
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0569
Step 700/1500, Loss: 0.0602
Step 800/1500, Loss: 0.0559
Step 900/1500, Loss: 0.0519
Step 1000/1500, Loss: 0.0606
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0582
Step 1200/1500, Loss: 0.0582
Step 1300/1500, Loss: 0.0577
Step 1400/1500, Loss: 0.0608

Training - Epoch 20 average loss: 0.0572
Running validation...
Validation - Epoch 20 average loss: 0.0564
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0564)

Epoch 21/30

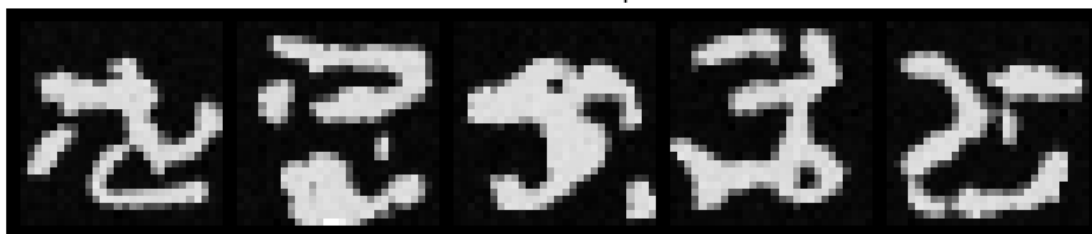
Step 0/1500, Loss: 0.0494
Step 100/1500, Loss: 0.0547
Step 200/1500, Loss: 0.0510
Step 300/1500, Loss: 0.0726
Step 400/1500, Loss: 0.0645
Step 500/1500, Loss: 0.0549
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0500
Step 700/1500, Loss: 0.0568
Step 800/1500, Loss: 0.0743
Step 900/1500, Loss: 0.0521
Step 1000/1500, Loss: 0.0603
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0533
Step 1200/1500, Loss: 0.0522
Step 1300/1500, Loss: 0.0548
Step 1400/1500, Loss: 0.0592

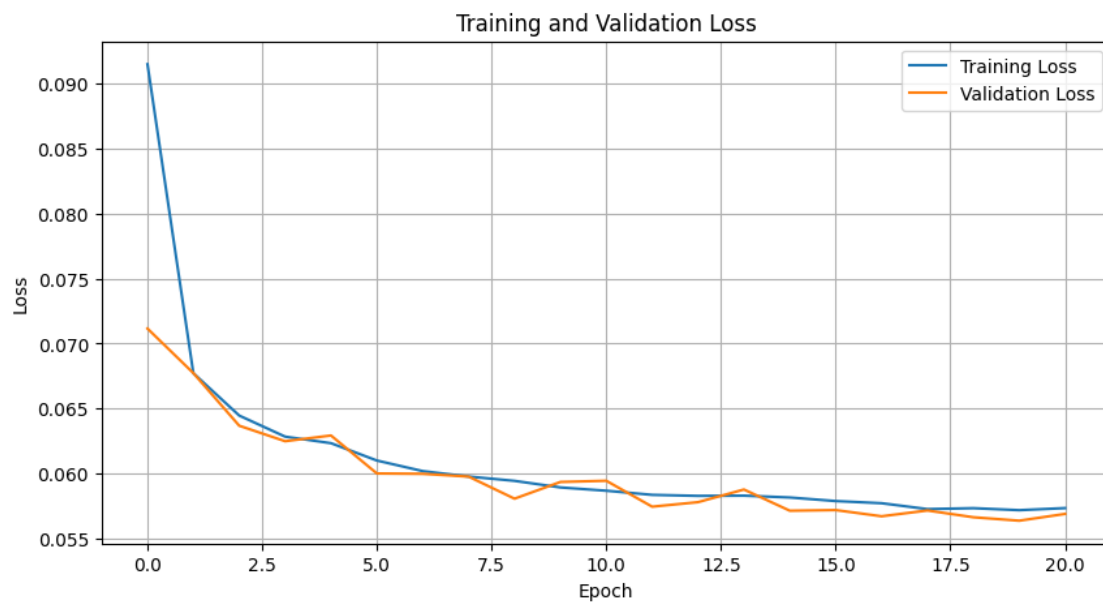
Training - Epoch 21 average loss: 0.0573
Running validation...
Validation - Epoch 21 average loss: 0.0569
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



No improvement for 1/10 epochs

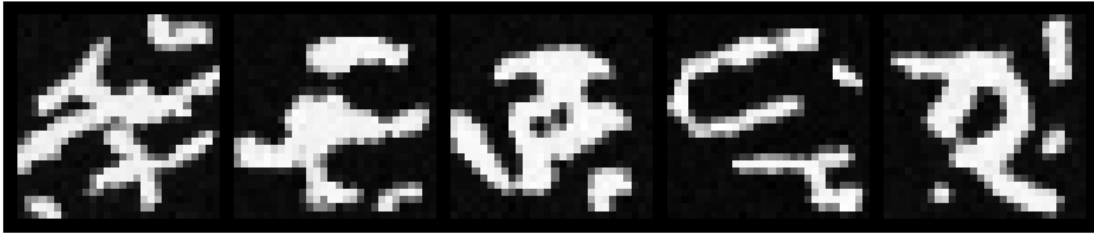


Epoch 22/30

Step 0/1500, Loss: 0.0635
Step 100/1500, Loss: 0.0547
Step 200/1500, Loss: 0.0562
Step 300/1500, Loss: 0.0522
Step 400/1500, Loss: 0.0602

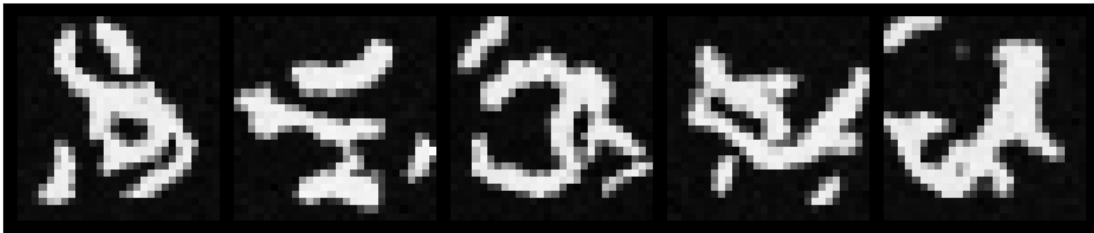
Step 500/1500, Loss: 0.0598
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0527
Step 700/1500, Loss: 0.0486
Step 800/1500, Loss: 0.0598
Step 900/1500, Loss: 0.0459
Step 1000/1500, Loss: 0.0707
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0622
Step 1200/1500, Loss: 0.0532
Step 1300/1500, Loss: 0.0653
Step 1400/1500, Loss: 0.0672

Training - Epoch 22 average loss: 0.0569
Running validation...
Validation - Epoch 22 average loss: 0.0570
Learning rate: 0.001000
No improvement for 2/10 epochs

Epoch 23/30

Step 0/1500, Loss: 0.0533
Step 100/1500, Loss: 0.0548
Step 200/1500, Loss: 0.0587

Step 300/1500, Loss: 0.0529
Step 400/1500, Loss: 0.0559
Step 500/1500, Loss: 0.0549
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0481
Step 700/1500, Loss: 0.0578
Step 800/1500, Loss: 0.0510
Step 900/1500, Loss: 0.0744
Step 1000/1500, Loss: 0.0543
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0566
Step 1200/1500, Loss: 0.0629
Step 1300/1500, Loss: 0.0561
Step 1400/1500, Loss: 0.0565

Training - Epoch 23 average loss: 0.0571
Running validation...
Validation - Epoch 23 average loss: 0.0568
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



No improvement for 3/10 epochs

Epoch 24/30

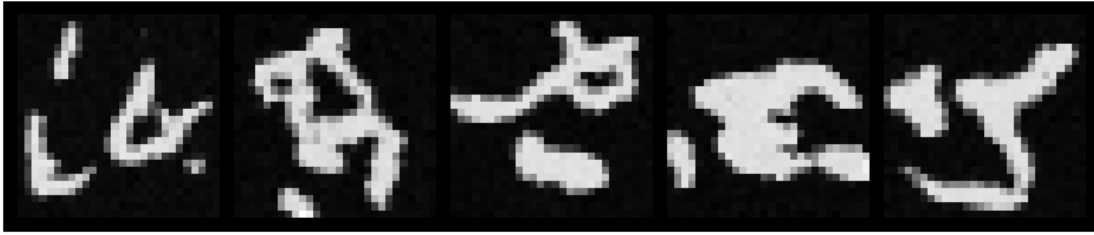
Step 0/1500, Loss: 0.0531
Step 100/1500, Loss: 0.0543
Step 200/1500, Loss: 0.0478
Step 300/1500, Loss: 0.0502
Step 400/1500, Loss: 0.0576
Step 500/1500, Loss: 0.0542
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0603
Step 700/1500, Loss: 0.0473
Step 800/1500, Loss: 0.0580
Step 900/1500, Loss: 0.0596
Step 1000/1500, Loss: 0.0660
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0619
Step 1200/1500, Loss: 0.0632
Step 1300/1500, Loss: 0.0627
Step 1400/1500, Loss: 0.0612

Training - Epoch 24 average loss: 0.0572
Running validation...
Validation - Epoch 24 average loss: 0.0563
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0563)

Epoch 25/30

Step 0/1500, Loss: 0.0595
Step 100/1500, Loss: 0.0700
Step 200/1500, Loss: 0.0520
Step 300/1500, Loss: 0.0554
Step 400/1500, Loss: 0.0562
Step 500/1500, Loss: 0.0613
Generating samples...

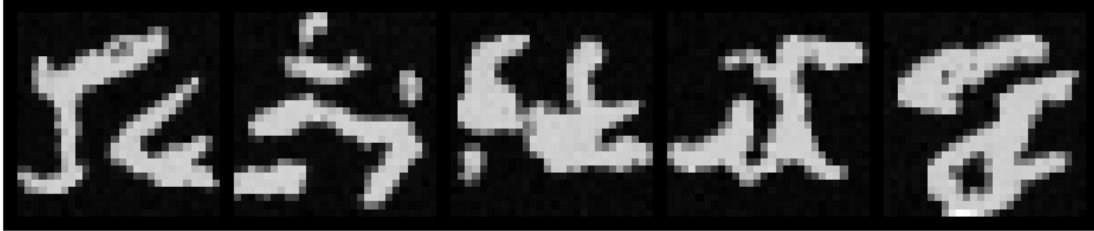
Generated Samples



Step 600/1500, Loss: 0.0493
Step 700/1500, Loss: 0.0496

Step 800/1500, Loss: 0.0528
Step 900/1500, Loss: 0.0529
Step 1000/1500, Loss: 0.0585
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0538
Step 1200/1500, Loss: 0.0515
Step 1300/1500, Loss: 0.0654
Step 1400/1500, Loss: 0.0616

Training - Epoch 25 average loss: 0.0567
Running validation...
Validation - Epoch 25 average loss: 0.0570
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

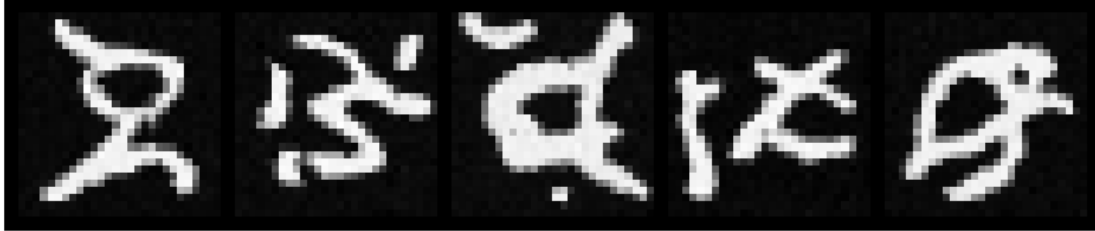


No improvement for 1/10 epochs

Epoch 26/30

Step 0/1500, Loss: 0.0632
Step 100/1500, Loss: 0.0571
Step 200/1500, Loss: 0.0537
Step 300/1500, Loss: 0.0524
Step 400/1500, Loss: 0.0519
Step 500/1500, Loss: 0.0562
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0577
Step 700/1500, Loss: 0.0544
Step 800/1500, Loss: 0.0628
Step 900/1500, Loss: 0.0584
Step 1000/1500, Loss: 0.0589
Generating samples...

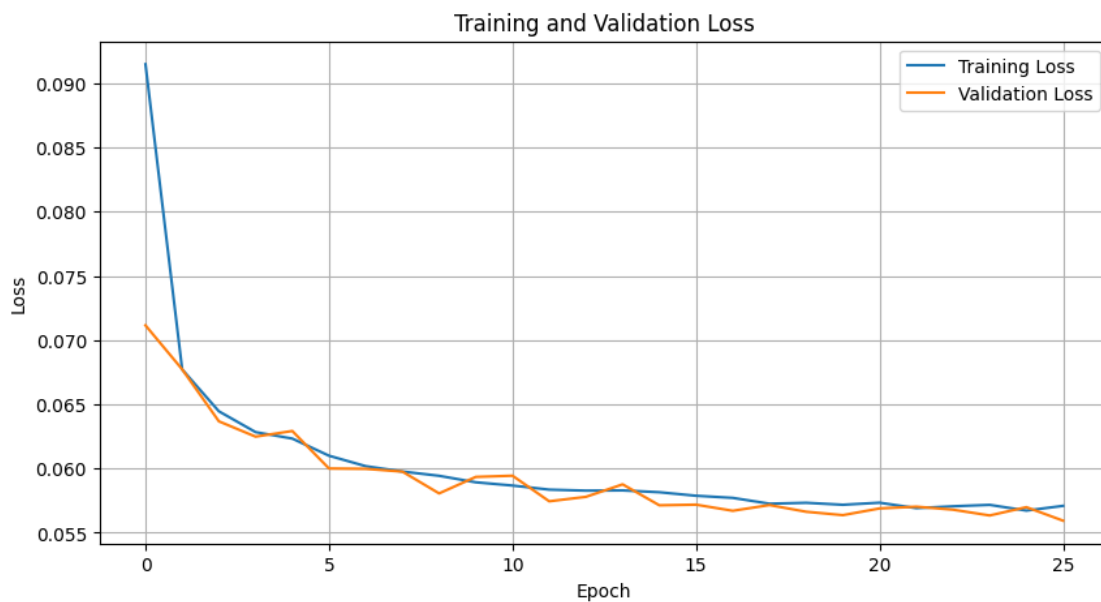
Generated Samples



Step 1100/1500, Loss: 0.0517
Step 1200/1500, Loss: 0.0538
Step 1300/1500, Loss: 0.0585
Step 1400/1500, Loss: 0.0668

Training - Epoch 26 average loss: 0.0571
Running validation...
Validation - Epoch 26 average loss: 0.0559
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt
New best model saved! (Val Loss: 0.0559)



Epoch 27/30

Step 0/1500, Loss: 0.0504
Step 100/1500, Loss: 0.0478
Step 200/1500, Loss: 0.0526
Step 300/1500, Loss: 0.0612
Step 400/1500, Loss: 0.0701
Step 500/1500, Loss: 0.0625
Generating samples...

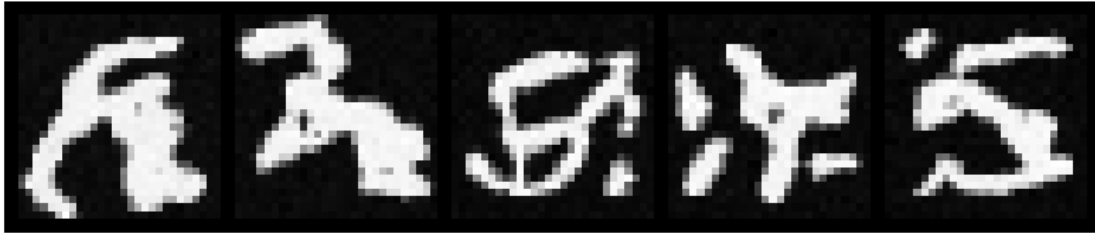
Generated Samples



Step 600/1500, Loss: 0.0515
Step 700/1500, Loss: 0.0535
Step 800/1500, Loss: 0.0685

Step 900/1500, Loss: 0.0566
Step 1000/1500, Loss: 0.0688
Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0562
Step 1200/1500, Loss: 0.0461
Step 1300/1500, Loss: 0.0634
Step 1400/1500, Loss: 0.0680

Training - Epoch 27 average loss: 0.0567
Running validation...
Validation - Epoch 27 average loss: 0.0560
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

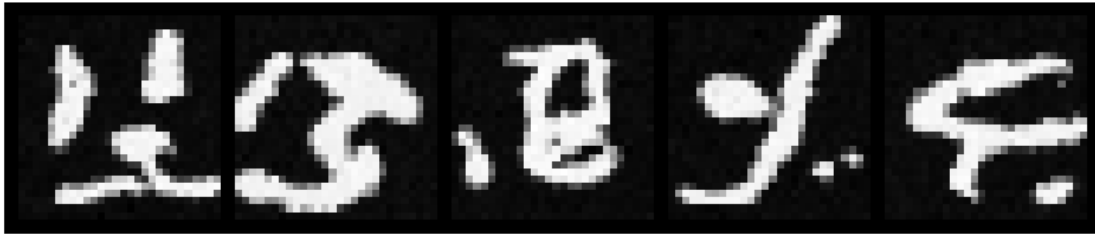


No improvement for 1/10 epochs

Epoch 28/30

Step 0/1500, Loss: 0.0551
Step 100/1500, Loss: 0.0570
Step 200/1500, Loss: 0.0649
Step 300/1500, Loss: 0.0599
Step 400/1500, Loss: 0.0503
Step 500/1500, Loss: 0.0570
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0539
Step 700/1500, Loss: 0.0563
Step 800/1500, Loss: 0.0463
Step 900/1500, Loss: 0.0628
Step 1000/1500, Loss: 0.0516
Generating samples...

Generated Samples



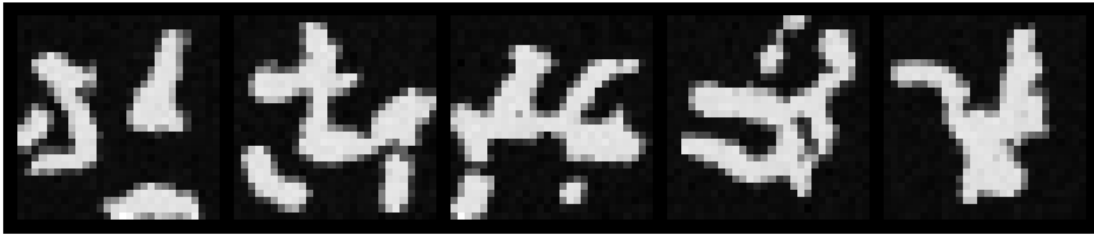
Step 1100/1500, Loss: 0.0570
Step 1200/1500, Loss: 0.0596
Step 1300/1500, Loss: 0.0560
Step 1400/1500, Loss: 0.0587

Training - Epoch 28 average loss: 0.0567
Running validation...
Validation - Epoch 28 average loss: 0.0562
Learning rate: 0.001000
No improvement for 2/10 epochs

Epoch 29/30

Step 0/1500, Loss: 0.0503
Step 100/1500, Loss: 0.0550
Step 200/1500, Loss: 0.0655
Step 300/1500, Loss: 0.0534
Step 400/1500, Loss: 0.0572
Step 500/1500, Loss: 0.0508
Generating samples...

Generated Samples



Step 600/1500, Loss: 0.0421
Step 700/1500, Loss: 0.0599
Step 800/1500, Loss: 0.0573
Step 900/1500, Loss: 0.0507
Step 1000/1500, Loss: 0.0533
Generating samples...

Generated Samples

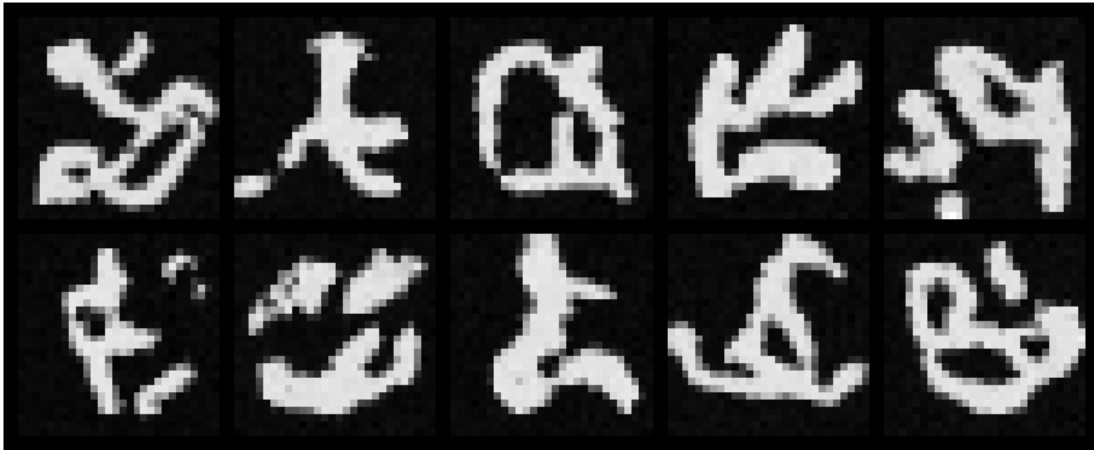


Step 1100/1500, Loss: 0.0505
Step 1200/1500, Loss: 0.0499
Step 1300/1500, Loss: 0.0543
Step 1400/1500, Loss: 0.0669

Training - Epoch 29 average loss: 0.0566
Running validation...
Validation - Epoch 29 average loss: 0.0568
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



No improvement for 3/10 epochs

Epoch 30/30

Step 0/1500, Loss: 0.0588
Step 100/1500, Loss: 0.0518
Step 200/1500, Loss: 0.0535
Step 300/1500, Loss: 0.0624
Step 400/1500, Loss: 0.0575
Step 500/1500, Loss: 0.0576
Generating samples...

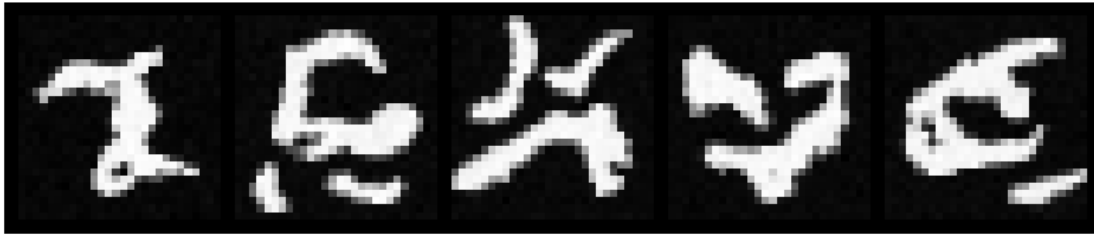
Generated Samples



Step 600/1500, Loss: 0.0663
Step 700/1500, Loss: 0.0553
Step 800/1500, Loss: 0.0641
Step 900/1500, Loss: 0.0523
Step 1000/1500, Loss: 0.0564

Generating samples...

Generated Samples



Step 1100/1500, Loss: 0.0589

Step 1200/1500, Loss: 0.0490

Step 1300/1500, Loss: 0.0620

Step 1400/1500, Loss: 0.0519

Training - Epoch 30 average loss: 0.0565

Running validation...

Validation - Epoch 30 average loss: 0.0557

Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



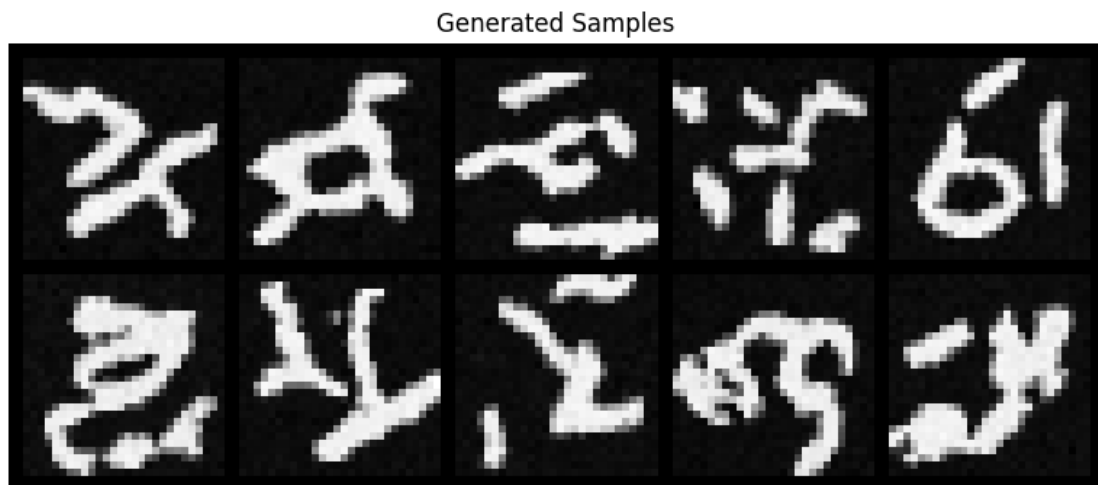
Created backup at best_diffusion_model.pt.backup

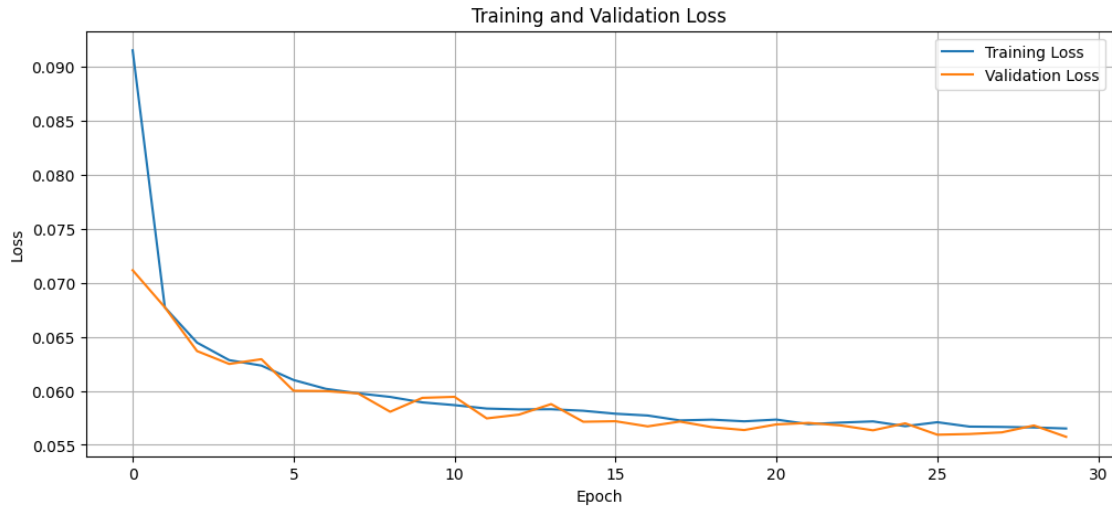
Model successfully saved to best_diffusion_model.pt

New best model saved! (Val Loss: 0.0557)



=====
TRAINING COMPLETE
=====
Best validation loss: 0.0557
Generating final samples...





```
[ ]: # Test the model with a dummy input to check dimensions
dummy_input = torch.randn(2, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)
dummy_t = torch.randint(0, n_steps, (2,)).to(device)
dummy_c = F.one_hot(torch.tensor([0, 1]), N_CLASSES).float().to(device)
dummy_mask = torch.ones(2, 1).to(device)

print("Testing model with dummy input...")
try:
    output = model(dummy_input, dummy_t, dummy_c, dummy_mask)
    print(f" Model test passed! Output shape: {output.shape}")
except Exception as e:
    print(f" Model test failed: {e}")
    import traceback
    traceback.print_exc()
```

```
[ ]: # Plot training progress
plt.figure(figsize=(12, 5))

# Plot training and validation losses for comparison
plt.plot(train_losses, label='Training Loss')
if len(val_losses) > 0: # Only plot validation if it exists
    plt.plot(val_losses, label='Validation Loss')

# Improve the plot with better labels and styling
plt.title('Diffusion Model Training Progress')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)
```

```

# Add annotations for key points
if len(train_losses) > 1:
    min_train_idx = train_losses.index(min(train_losses))
    plt.annotate(f'Min: {min(train_losses):.4f}',
                 xy=(min_train_idx, min(train_losses)),
                 xytext=(min_train_idx, min(train_losses)*1.2),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)

# Add validation min point if available
if len(val_losses) > 1:
    min_val_idx = val_losses.index(min(val_losses))
    plt.annotate(f'Min: {min(val_losses):.4f}',
                 xy=(min_val_idx, min(val_losses)),
                 xytext=(min_val_idx, min(val_losses)*0.8),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)

# Set y-axis to start from 0 or slightly lower than min value
plt.ylim(bottom=max(0, min(min(train_losses) if train_losses else float('inf'),
                           min(val_losses) if val_losses else float('inf'))*0.9))

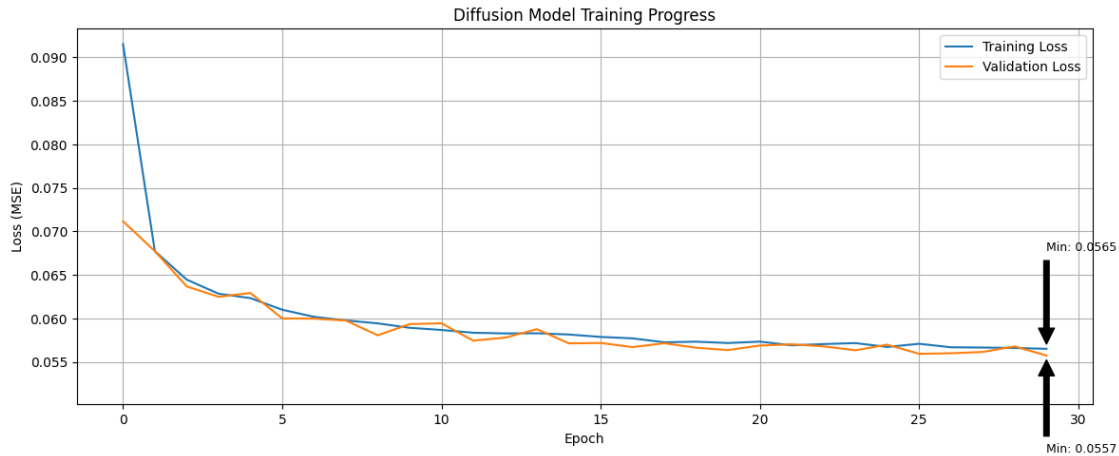
plt.tight_layout()
plt.show()

# Add statistics summary for students to analyze
print("\nTraining Statistics:")
print("-" * 30)
if train_losses:
    print(f"Starting training loss:      {train_losses[0]:.4f}")
    print(f"Final training loss:              {train_losses[-1]:.4f}")
    print(f"Best training loss:                {min(train_losses):.4f}")
    print(f"Training loss improvement: {((train_losses[0] - min(train_losses)) /
    ↪ train_losses[0] * 100):.1f}%")

if val_losses:
    print("\nValidation Statistics:")
    print("-" * 30)
    print(f"Starting validation loss: {val_losses[0]:.4f}")
    print(f"Final validation loss:    {val_losses[-1]:.4f}")
    print(f"Best validation loss:     {min(val_losses):.4f}")

# STUDENT EXERCISE:
# 1. Try modifying this plot to show a smoothed version of the losses
# 2. Create a second plot showing the ratio of validation to training loss
#    (which can indicate overfitting when the ratio increases)

```



Training Statistics:

```
-----
Starting training loss:    0.0915
Final training loss:      0.0565
Best training loss:       0.0565
Training loss improvement: 38.2%
```

Validation Statistics:

```
-----
Starting validation loss: 0.0712
Final validation loss:    0.0557
Best validation loss:     0.0557
```

1.7 Step 6: Generating New Images

Now that our model is trained, let's generate some new images! We can: 1. Generate specific numbers 2. Generate multiple versions of each number 3. See how the generation process works step by step

```
[ ]: def generate_number(model, number, n_samples=4):
      """
      Generate multiple versions of a specific number using the diffusion model.

      Args:
          model (nn.Module): The trained diffusion model
          number (int): The digit to generate (0-9)
          n_samples (int): Number of variations to generate

      Returns:
          torch.Tensor: Generated images of shape [n_samples, IMG_CH, IMG_SIZE, 3]
          ↳ IMG_SIZE]
```

```

"""
model.eval() # Set model to evaluation mode
with torch.no_grad(): # No need for gradients during generation
    # Start with random noise
    samples = torch.randn(n_samples, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

    # Set up the number we want to generate
    c = torch.full((n_samples,), number).to(device)
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
    # Correctly sized conditioning mask
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Display progress information
    print(f"Generating {n_samples} versions of number {number}...")

    # Remove noise step by step
    for t in range(n_steps-1, -1, -1):
        t_batch = torch.full((n_samples,), t).to(device)
        samples = remove_noise(samples, t_batch, model, c_one_hot, c_mask)

        # Optional: Display occasional progress updates
        if t % (n_steps // 5) == 0:
            print(f"  Denoising step {n_steps-1-t}/{n_steps-1} completed")

    return samples

# Generate 4 versions of each number
plt.figure(figsize=(20, 10))
for i in range(10):
    # Generate samples for current digit
    samples = generate_number(model, i, n_samples=4)

    # Display each sample
    for j in range(4):
        # Use 2 rows, 10 digits per row, 4 samples per digit
        # i//5 determines the row (0 or 1)
        # i%5 determines the position in the row (0-4)
        # j is the sample index within each digit (0-3)
        plt.subplot(5, 8, (i%5)*8 + (i//5)*4 + j + 1)

        # Display the image correctly based on channel configuration
        if IMG_CH == 1: # Grayscale
            plt.imshow(samples[j][0].cpu(), cmap='gray')
        else: # Color image
            img = samples[j].permute(1, 2, 0).cpu()
            # Rescale from [-1, 1] to [0, 1] if needed
            if img.min() < 0:

```

```

        img = (img + 1) / 2
        plt.imshow(img)

        plt.title(f'Digit {i}')
        plt.axis('off')

plt.tight_layout()
plt.show()

# STUDENT ACTIVITY: Try generating the same digit with different noise seeds
# This shows the variety of styles the model can produce
print("\nSTUDENT ACTIVITY: Generating numbers with different noise seeds")

# Helper function to generate with seed
def generate_with_seed(number, seed_value=42, n_samples=10):
    torch.manual_seed(seed_value)
    return generate_number(model, number, n_samples)

# Pick a image and show many variations
# Hint select a image e.g. dog # Change this to any other in the dataset of 
↳ subset you chose
# Hint 2 use variations = generate_with_seed
# Hint 3 use plt.figure and plt.imshow to display the variations

# Enter your code here:

```

Generating 4 versions of number 0...

```

Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed

```

Generating 4 versions of number 1...

```

Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed

```

Generating 4 versions of number 2...

```

Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed

```

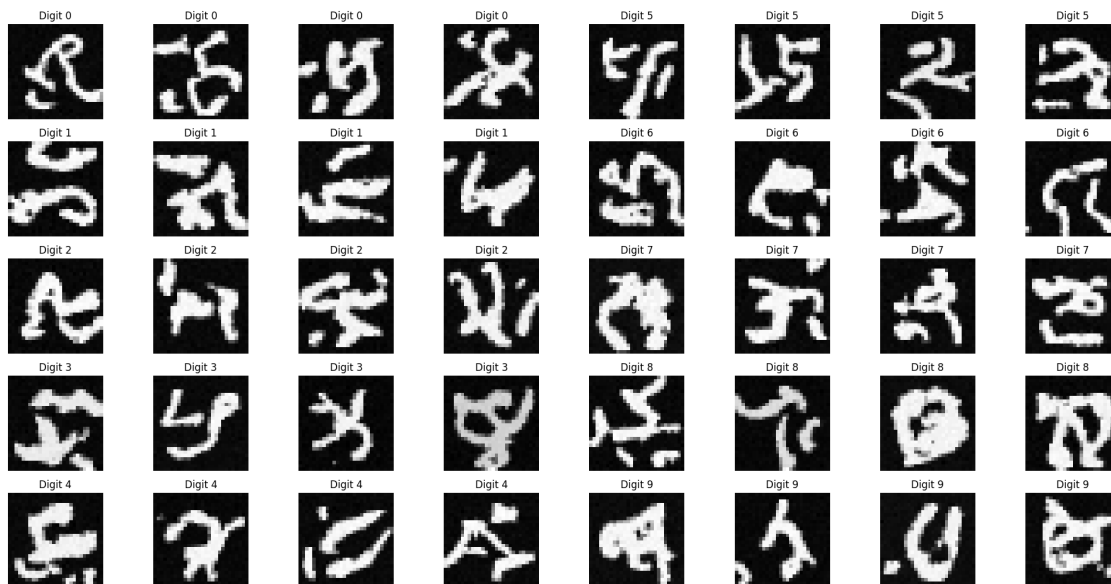
Generating 4 versions of number 3...

```

Denoising step 19/99 completed
Denoising step 39/99 completed

```

Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 4...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 5...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 6...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 7...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 8...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed
Generating 4 versions of number 9...
Denoising step 19/99 completed
Denoising step 39/99 completed
Denoising step 59/99 completed
Denoising step 79/99 completed
Denoising step 99/99 completed



STUDENT ACTIVITY: Generating numbers with different noise seeds

1.8 Step 7: Watching the Generation Process

Let's see how our model turns random noise into clear images, step by step. This helps us understand how the diffusion process works!

```
[ ]: def visualize_generation_steps(model, number, n_preview_steps=10):
    """
    Show how an image evolves from noise to a clear number
    """
    model.eval()
    with torch.no_grad():
        # Start with random noise
        x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

        # Set up which number to generate
        c = torch.tensor([number]).to(device)
        c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
        c_mask = torch.ones(1, 1).to(device) # torch.ones_like(c_one_hot).
        ↪to(device)

        # Calculate which steps to show
        steps_to_show = torch.linspace(n_steps-1, 0, n_preview_steps).long()

        # Store images for visualization
        images = []
```



```

images.append(x[0].cpu())

# Remove noise step by step
for t in range(n_steps-1, -1, -1):
    t_batch = torch.full((1,), t).to(device)
    x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

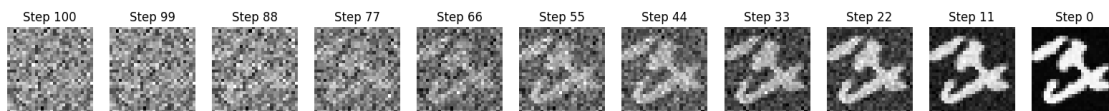
    if t in steps_to_show:
        images.append(x[0].cpu())

# Show the progression
plt.figure(figsize=(20, 3))
for i, img in enumerate(images):
    plt.subplot(1, len(images), i+1)
    if IMG_CH == 1:
        plt.imshow(img[0], cmap='gray')
    else:
        img = img.permute(1, 2, 0)
        if img.min() < 0:
            img = (img + 1) / 2
        plt.imshow(img)
    step = n_steps if i == 0 else steps_to_show[i-1]
    plt.title(f'Step {step}')
    plt.axis('off')
plt.show()

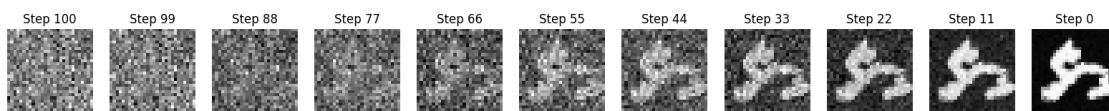
# Show generation process for a few numbers
for number in [0, 3, 7]:
    print(f"\nGenerating number {number}:")
    visualize_generation_steps(model, number)

```

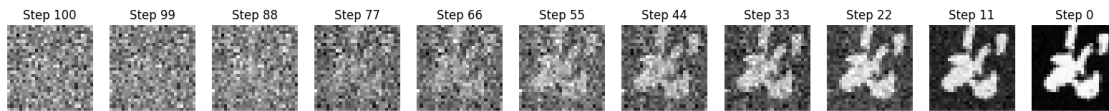
Generating number 0:



Generating number 3:



Generating number 7:



1.9 Step 8: Adding CLIP Evaluation

CLIP is a powerful AI model that can understand both images and text. We'll use it to: 1. Evaluate how realistic our generated images are 2. Score how well they match their intended numbers 3. Help guide the generation process towards better quality

```
[ ]: ## Step 8: Adding CLIP Evaluation

# CLIP (Contrastive Language-Image Pre-training) is a powerful model by OpenAI
# that connects text and images.
# We'll use it to evaluate how recognizable our generated digits are by
# measuring how strongly
# the CLIP model associates our generated images with text descriptions like
# "an image of the digit 7".

# First, we need to install CLIP and its dependencies
print("Setting up CLIP (Contrastive Language-Image Pre-training) model...")

# Track installation status
clip_available = False

try:
    # Install dependencies first - these help CLIP process text and images
    print("Installing CLIP dependencies...")
    !pip install -q ftfy regex tqdm

    # Install CLIP from GitHub
    print("Installing CLIP from GitHub repository...")
    !pip install -q git+https://github.com/openai/CLIP.git

    # Import and verify CLIP is working
    print("Importing CLIP...")
    import clip

    # Test that CLIP is functioning
    models = clip.available_models()
    print(f" CLIP installation successful! Available models: {models}")
```

```

clip_available = True

except ImportError:
    print(" Error importing CLIP. Installation might have failed.")
    print("Try manually running: !pip install git+https://github.com/openai/
↳CLIP.git")
    print("If you're in a Colab notebook, try restarting the runtime after
↳installation.")

except Exception as e:
    print(f" Error during CLIP setup: {e}")
    print("Some CLIP functionality may not work correctly.")

# Provide guidance based on installation result
if clip_available:
    print("\nCLIP is now available for evaluating your generated images!")
else:
    print("\nWARNING: CLIP installation failed. We'll skip the CLIP evaluation
↳parts.")

# Import necessary libraries
import functools
import torch.nn.functional as F

```

Setting up CLIP (Contrastive Language-Image Pre-training) model...

Installing CLIP dependencies...

44.8/44.8 kB

4.3 MB/s eta 0:00:00

Installing CLIP from GitHub repository...

Preparing metadata (setup.py) ... done

Building wheel for clip (setup.py) ... done

Importing CLIP...

CLIP installation successful! Available models: ['RN50', 'RN101', 'RN50x4', 'RN50x16', 'RN50x64', 'ViT-B/32', 'ViT-B/16', 'ViT-L/14', 'ViT-L/14@336px']

CLIP is now available for evaluating your generated images!

Below we are creating a helper function to manage GPU memory when using CLIP. CLIP can be memory-intensive, so this will help prevent out-of-memory errors:

```

[ ]: # Memory management decorator to prevent GPU OOM errors
def manage_gpu_memory(func):
    """
    Decorator that ensures proper GPU memory management.

    This wraps functions that might use large amounts of GPU memory,
    making sure memory is properly freed after function execution.
    """

```

```

@functools.wraps(func)
def wrapper(*args, **kwargs):
    if torch.cuda.is_available():
        # Clear cache before running function
        torch.cuda.empty_cache()
    try:
        return func(*args, **kwargs)
    finally:
        # Clear cache after running function regardless of success/
        ↪ failure
        torch.cuda.empty_cache()
    return func(*args, **kwargs)
return wrapper

```

```

[ ]: #=====
# Step 8: CLIP Model Loading and Evaluation Setup
#=====
# CLIP (Contrastive Language-Image Pre-training) is a neural network that
    ↪ connects
# vision and language. It was trained on 400 million image-text pairs to
    ↪ understand
# the relationship between images and their descriptions.
# We use it here as an "evaluation judge" to assess our generated images.

# Load CLIP model with error handling
try:
    # Load the ViT-B/32 CLIP model (Vision Transformer-based)
    clip_model, clip_preprocess = clip.load("ViT-B/32", device=device)
    print(f" Successfully loaded CLIP model: {clip_model.visual.__class___.
    ↪ __name__}")
except Exception as e:
    print(f" Failed to load CLIP model: {e}")
    clip_available = False
    # Instead of raising an error, we'll continue with degraded functionality
    print("CLIP evaluation will be skipped. Generated images will still be
    ↪ displayed but without quality scores.")

def evaluate_with_clip(images, target_number, max_batch_size=16):
    """
    Use CLIP to evaluate generated images by measuring how well they match
    ↪ textual descriptions.

    This function acts like an "automatic critic" for our generated digits by
    ↪ measuring:
    1. How well they match the description of a handwritten digit
    2. How clear and well-formed they appear to be

```

3. Whether they appear blurry or poorly formed

The evaluation process works by:

- Converting our images to a format CLIP understands
- Creating text prompts that describe the qualities we want to measure
- Computing similarity scores between images and these text descriptions
- Returning normalized scores (probabilities) for each quality

Args:

`images (torch.Tensor)`: Batch of generated images [batch_size, channels, height, width]
`target_number (int)`: The specific digit (0-9) the images should represent
`max_batch_size (int)`: Maximum images to process at once (prevents GPU out-of-memory errors)

Returns:

`torch.Tensor`: Similarity scores tensor of shape [batch_size, 3] with scores for:

[good handwritten digit, clear digit, blurry digit]
Each row sums to 1.0 (as probabilities)

```
"""
# If CLIP isn't available, return placeholder scores
if not clip_available:
    print(" CLIP not available. Returning default scores.")
    # Equal probabilities (0.33 for each category)
    return torch.ones(len(images), 3).to(device) / 3

try:
    # For large batches, we process in chunks to avoid memory issues
    # This is crucial when working with big images or many samples
    if len(images) > max_batch_size:
        all_similarities = []

        # Process images in manageable chunks
        for i in range(0, len(images), max_batch_size):
            print(f"Processing CLIP batch {i//max_batch_size + 1}/
            {(len(images)-1)//max_batch_size + 1}")
            batch = images[i:i+max_batch_size]

            # Use context managers for efficiency and memory management:
            # - torch.no_grad(): disables gradient tracking (not needed for
            evaluation)
            # - torch.cuda.amp.autocast(): uses mixed precision to reduce
            memory usage
            with torch.no_grad(), torch.cuda.amp.autocast():
```

```

        batch_similarities = _process_clip_batch(batch,
↪target_number)

        all_similarities.append(batch_similarities)

        # Explicitly free GPU memory between batches
        # This helps prevent cumulative memory buildup that could cause
↪crashes

        torch.cuda.empty_cache()

        # Combine results from all batches into a single tensor
        return torch.cat(all_similarities, dim=0)
    else:
        # For small batches, process all at once
        with torch.no_grad(), torch.cuda.amp.autocast():
            return _process_clip_batch(images, target_number)

except Exception as e:
    # If anything goes wrong, log the error but don't crash
    print(f" Error in CLIP evaluation: {e}")
    print(f"Traceback: {traceback.format_exc()}")
    # Return default scores so the rest of the notebook can continue
    return torch.ones(len(images), 3).to(device) / 3

def _process_clip_batch(images, target_number):
    """
    Core CLIP processing function that computes similarity between images and
↪text descriptions.

    This function handles the technical details of:
    1. Preparing relevant text prompts for evaluation
    2. Preprocessing images to CLIP's required format
    3. Extracting feature embeddings from both images and text
    4. Computing similarity scores between these embeddings

    The function includes advanced error handling for GPU memory issues,
    automatically reducing batch size if out-of-memory errors occur.

    Args:
        images (torch.Tensor): Batch of images to evaluate
        target_number (int): The digit these images should represent

    Returns:
        torch.Tensor: Normalized similarity scores between images and text
↪descriptions
    """
    try:
        # Create text descriptions (prompts) to evaluate our generated digits

```

```

# We check three distinct qualities:
# 1. If it looks like a handwritten example of the target digit
# 2. If it appears clear and well-formed
# 3. If it appears blurry or poorly formed (negative case)
text_inputs = torch.cat([
    clip.tokenize(f"A handwritten number {target_number}"),
    clip.tokenize(f"A clear, well-written digit {target_number}"),
    clip.tokenize(f"A blurry or unclear number")
]).to(device)

# Process images for CLIP, which requires specific formatting:

# 1. Handle different channel configurations (dataset-dependent)
if IMG_CH == 1:
    # CLIP expects RGB images, so we repeat the grayscale channel 3
    ↪times
    # For example, MNIST/Fashion-MNIST are grayscale (1-channel)
    images_rgb = images.repeat(1, 3, 1, 1)
else:
    # For RGB datasets like CIFAR-10/CelebA, we can use as-is
    images_rgb = images

# 2. Normalize pixel values to [0,1] range if needed
# Different datasets may have different normalization ranges
if images_rgb.min() < 0: # If normalized to [-1,1] range
    images_rgb = (images_rgb + 1) / 2 # Convert to [0,1] range

# 3. Resize images to CLIP's expected input size (224x224 pixels)
# CLIP was trained on this specific resolution
resized_images = F.interpolate(images_rgb, size=(224, 224),
                                mode='bilinear', align_corners=False)

# Extract feature embeddings from both images and text prompts
# These are high-dimensional vectors representing the content
image_features = clip_model.encode_image(resized_images)
text_features = clip_model.encode_text(text_inputs)

# Normalize feature vectors to unit length (for cosine similarity)
# This ensures we're measuring direction, not magnitude
image_features = image_features / image_features.norm(dim=-1,
    ↪keepdim=True)
text_features = text_features / text_features.norm(dim=-1, keepdim=True)

# Calculate similarity scores between image and text features
# The matrix multiplication computes all pairwise dot products at once
# Multiplying by 100 scales to percentage-like values before applying
    ↪softmax

```

```

        similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)

    return similarity

except RuntimeError as e:
    # Special handling for CUDA out-of-memory errors
    if "out of memory" in str(e):
        # Free GPU memory immediately
        torch.cuda.empty_cache()

        # If we're already at batch size 1, we can't reduce further
        if len(images) <= 1:
            print(" Out of memory even with batch size 1. Cannot process.")
            return torch.ones(len(images), 3).to(device) / 3

        # Adaptive batch size reduction - recursively try with smaller
    ↪batches
        # This is an advanced technique to handle limited GPU memory
    ↪gracefully
        half_size = len(images) // 2
        print(f" Out of memory. Reducing batch size to {half_size}.")

        # Process each half separately and combine results
        # This recursive approach will keep splitting until processing
    ↪succeeds
        first_half = _process_clip_batch(images[:half_size], target_number)
        second_half = _process_clip_batch(images[half_size:], target_number)

        # Combine results from both halves
        return torch.cat([first_half, second_half], dim=0)

    # For other errors, propagate upward
    raise e

=====
# CLIP Evaluation - Generate and Analyze Sample Digits
=====
# This section demonstrates how to use CLIP to evaluate generated digits
# We'll generate examples of all ten digits and visualize the quality scores

try:
    for number in range(10):
        print(f"\nGenerating and evaluating number {number}...")

        # Generate 4 different variations of the current digit
        samples = generate_number(model, number, n_samples=4)

```



```

# Evaluate quality with CLIP (without tracking gradients for efficiency)
with torch.no_grad():
    similarities = evaluate_with_clip(samples, number)

# Create a figure to display results
plt.figure(figsize=(15, 3))

# Show each sample with its CLIP quality scores
for i in range(4):
    plt.subplot(1, 4, i+1)

    # Display the image with appropriate formatting based on dataset
    ↪type
    if IMG_CH == 1: # Grayscale images (MNIST, Fashion-MNIST)
        plt.imshow(samples[i][0].cpu(), cmap='gray')
    else: # Color images (CIFAR-10, CelebA)
        img = samples[i].permute(1, 2, 0).cpu() # Change format for
        ↪matplotlib
        if img.min() < 0: # Handle [-1,1] normalization
            img = (img + 1) / 2 # Convert to [0,1] range
        plt.imshow(img)

    # Extract individual quality scores for display
    # These represent how confidently CLIP associates the image with
    ↪each description
    good_score = similarities[i][0].item() * 100 # Handwritten quality
    clear_score = similarities[i][1].item() * 100 # Clarity quality
    blur_score = similarities[i][2].item() * 100 # Blurriness
    ↪assessment

    # Color-code the title based on highest score category:
    # - Green: if either "good handwritten" or "clear" score is highest
    # - Red: if "blurry" score is highest (poor quality)
    max_score_idx = torch.argmax(similarities[i]).item()
    title_color = 'green' if max_score_idx < 2 else 'red'

    # Show scores in the plot title
    plt.title(f'Number {number}\nGood: {good_score:.0f}%\nClear:
    ↪{clear_score:.0f}%\nBlurry: {blur_score:.0f}%',
              color=title_color)
    plt.axis('off')

plt.tight_layout()
plt.show()
plt.close() # Properly close figure to prevent memory leaks

# Clean up GPU memory after processing each number

```

```

        # This is especially important for resource-constrained environments
        torch.cuda.empty_cache()

except Exception as e:
    # Comprehensive error handling to help students debug issues
    print(f" Error in generation and evaluation loop: {e}")
    print("Detailed error information:")
    import traceback
    traceback.print_exc()

    # Clean up resources even if we encounter an error
    if torch.cuda.is_available():
        print("Clearing GPU cache...")
        torch.cuda.empty_cache()

#=====
# STUDENT ACTIVITY: Exploring CLIP Evaluation
#=====
# This section provides code templates for students to experiment with
# evaluating larger batches of generated digits using CLIP.

print("\nSTUDENT ACTIVITY:")
print("Try the code below to evaluate a larger sample of a specific digit")
print("""
# Example: Generate and evaluate 10 examples of the digit 6
# digit = 6
# samples = generate_number(model, digit, n_samples=10)
# similarities = evaluate_with_clip(samples, digit)
#
# # Calculate what percentage of samples CLIP considers "good quality"
# # (either "good handwritten" or "clear" score exceeds "blurry" score)
# good_or_clear = (similarities[:,0] + similarities[:,1] > similarities[:,2]).
    ↪float().mean()
# print(f"CLIP recognized {good_or_clear.item()*100:.1f}% of the digits as good,
    ↪examples of {digit}")
#
# # Display a grid of samples with their quality scores
# plt.figure(figsize=(15, 8))
# for i in range(len(samples)):
#     plt.subplot(2, 5, i+1)
#     plt.imshow(samples[i][0].cpu(), cmap='gray')
#     quality = "Good" if similarities[i,0] + similarities[i,1] >
    ↪similarities[i,2] else "Poor"
#     plt.title(f"Sample {i+1}: {quality}", color='green' if quality == "Good"
    ↪else 'red')
#     plt.axis('off')
# plt.tight_layout()

```

```
# plt.show()
""")
```

Successfully loaded CLIP model: VisionTransformer

Generating and evaluating number 0...

Generating 4 versions of number 0...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

/tmp/ipykernel_9502/3974156644.py:77: FutureWarning:

`torch.cuda.amp.autocast(args...)` is deprecated. Please use

`torch.amp.autocast('cuda', args...)` instead.

with torch.no_grad(), torch.cuda.amp.autocast():

Number 0
Good: 2%
Clear: 81%
Blurry: 18%



Number 0
Good: 2%
Clear: 48%
Blurry: 50%



Number 0
Good: 2%
Clear: 72%
Blurry: 26%



Number 0
Good: 2%
Clear: 61%
Blurry: 37%



Generating and evaluating number 1...

Generating 4 versions of number 1...

Denoising step 19/99 completed

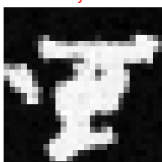
Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

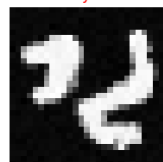
Number 1
Good: 0%
Clear: 4%
Blurry: 95%



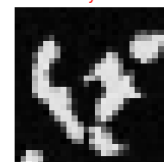
Number 1
Good: 0%
Clear: 3%
Blurry: 97%



Number 1
Good: 1%
Clear: 24%
Blurry: 75%



Number 1
Good: 0%
Clear: 16%
Blurry: 84%



Generating and evaluating number 2...

Generating 4 versions of number 2...

Denoising step 19/99 completed

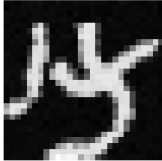
Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

Number 2
Good: 1%
Clear: 21%
Blurry: 78%



Number 2
Good: 0%
Clear: 19%
Blurry: 81%



Number 2
Good: 1%
Clear: 28%
Blurry: 70%



Number 2
Good: 1%
Clear: 16%
Blurry: 84%



Generating and evaluating number 3...

Generating 4 versions of number 3...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

Number 3
Good: 1%
Clear: 31%
Blurry: 69%



Number 3
Good: 0%
Clear: 5%
Blurry: 95%



Number 3
Good: 0%
Clear: 5%
Blurry: 95%



Number 3
Good: 0%
Clear: 2%
Blurry: 97%



Generating and evaluating number 4...

Generating 4 versions of number 4...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

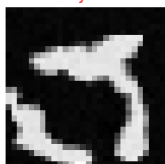
Denoising step 79/99 completed

Denoising step 99/99 completed

Number 4
Good: 1%
Clear: 81%
Blurry: 18%



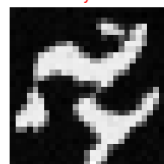
Number 4
Good: 0%
Clear: 10%
Blurry: 90%



Number 4
Good: 0%
Clear: 1%
Blurry: 99%



Number 4
Good: 0%
Clear: 11%
Blurry: 88%



Generating and evaluating number 5...

Generating 4 versions of number 5...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

Number 5
Good: 0%
Clear: 16%
Blurry: 84%



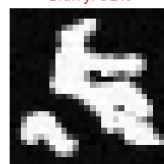
Number 5
Good: 0%
Clear: 8%
Blurry: 92%



Number 5
Good: 1%
Clear: 14%
Blurry: 86%



Number 5
Good: 0%
Clear: 8%
Blurry: 91%



Generating and evaluating number 6...

Generating 4 versions of number 6...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

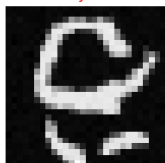
Denoising step 79/99 completed

Denoising step 99/99 completed

Number 6
Good: 1%
Clear: 54%
Blurry: 45%



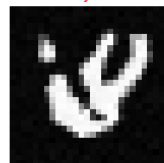
Number 6
Good: 1%
Clear: 47%
Blurry: 52%



Number 6
Good: 1%
Clear: 23%
Blurry: 76%



Number 6
Good: 0%
Clear: 31%
Blurry: 69%



Generating and evaluating number 7...

Generating 4 versions of number 7...

Denoising step 19/99 completed

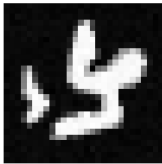
Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

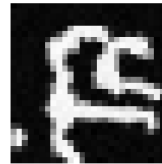
Number 7
Good: 0%
Clear: 12%
Blurry: 88%



Number 7
Good: 1%
Clear: 9%
Blurry: 91%



Number 7
Good: 1%
Clear: 17%
Blurry: 82%



Number 7
Good: 1%
Clear: 24%
Blurry: 75%



Generating and evaluating number 8...

Generating 4 versions of number 8...

Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed

Number 8
Good: 1%
Clear: 63%
Blurry: 36%



Number 8
Good: 0%
Clear: 6%
Blurry: 93%



Number 8
Good: 2%
Clear: 58%
Blurry: 40%



Number 8
Good: 0%
Clear: 3%
Blurry: 97%



Generating and evaluating number 9...

Generating 4 versions of number 9...

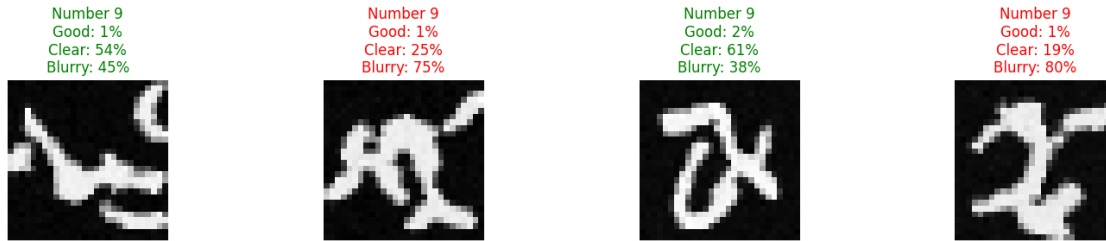
Denoising step 19/99 completed

Denoising step 39/99 completed

Denoising step 59/99 completed

Denoising step 79/99 completed

Denoising step 99/99 completed



STUDENT ACTIVITY:

Try the code below to evaluate a larger sample of a specific digit

```
# Example: Generate and evaluate 10 examples of the digit 6
# digit = 6
# samples = generate_number(model, digit, n_samples=10)
# similarities = evaluate_with_clip(samples, digit)
#
# # Calculate what percentage of samples CLIP considers "good quality"
# # (either "good handwritten" or "clear" score exceeds "blurry" score)
# good_or_clear = (similarities[:,0] + similarities[:,1] >
similarities[:,2]).float().mean()
# print(f"CLIP recognized {good_or_clear.item()*100:.1f}% of the digits as good
examples of {digit}")
#
# # Display a grid of samples with their quality scores
# plt.figure(figsize=(15, 8))
# for i in range(len(samples)):
#     plt.subplot(2, 5, i+1)
#     plt.imshow(samples[i][0].cpu(), cmap='gray')
#     quality = "Good" if similarities[i,0] + similarities[i,1] >
similarities[i,2] else "Poor"
#     plt.title(f"Sample {i+1}: {quality}", color='green' if quality == "Good"
else 'red')
#     plt.axis('off')
# plt.tight_layout()
# plt.show()
```

1.10 Assessment Questions

Now that you've completed the exercise, answer these questions include explanations, observations, and your analysis Support your answers with specific examples from your experiments:

1.10.1 1. Understanding Diffusion

- Explain what happens during the forward diffusion process, using your own words and referencing the visualization examples from your notebook. ANS: The forward diffusion process is where we add noise to the image. It works by starting off with a clean image (x_0), to which we gradually add Gaussian noise over (T) timesteps until it becomes pure noise at (x_T).
- Why do we add noise gradually instead of all at once? How does this affect the learning process? ANS: We add noise gradually so the model can learn to denoise in small, manageable steps rather than trying to reconstruct a clean image from pure static in one shot. At early timesteps, the model learns fine details, edges and texture, at mid timesteps, the model learns composition and overall shape, and at the late timesteps, the model learns general layout. With a dump of noise at once, these steps would not be possible.
- Look at the step-by-step visualization - at what point (approximately what percentage through the denoising process) can you first recognize the image? Does this vary by image? ANS: Typically the image becomes recognizable around 60-70% through the denoising process (around timestep 300-400 out of 1000). This does vary by digit; simpler digits like 1 and 7 tend to emerge earlier because they have less structural complexity, while digits like 8 and 9 with more complex curves take longer to become identifiable.

1.10.2 2. Model Architecture

- Why is the U-Net architecture particularly well-suited for diffusion models? What advantages does it provide over simpler architectures? ANS: U-Net is ideal because it operates at multiple spatial resolutions simultaneously. The encoder compresses the image to capture high-level semantic features, while the decoder decompresses for spatial details. This matters for diffusion because the model needs to understand both what the image contains (semantics) and exactly where each pixel should be (spatial precision). Simpler architectures like plain CNNs lack this multi-scale understanding.
- What are skip connections and why are they important? Explain them in relation to our model ANS: Skip connections are direct pathways that copy feature maps from the encoder to the corresponding decoder layer. In our model, they're the "skip_connections" list in `UNet.forward`. They're important because the downsampling path loses spatial information; skip connections restore it during upsampling, allowing the model to produce sharp, detailed outputs rather than blurry reconstructions.
- Describe in detail how our model is conditioned to generate specific images. How does the class conditioning mechanism work? ANS: Our model uses two conditioning signals. The class label is converted to a one-hot vector, passed through `EmbedBlock` to produce a `[B, 128, 1, 1]` embedding, then added directly to the feature map at the bottleneck: $x = x + t_emb + c_emb$. The `c_mask` controls whether conditioning is applied, setting it to 0 makes generation unconditional, which enables classifier-free guidance during inference.

1.10.3 3. Training Analysis (20 points)

- What does the loss value tell of your model tell us? ANS: The loss measures how accurately the model predicted the noise that was added to the image at each timestep. A decreasing training loss means the model is getting better at noise prediction. The gap between training

and validation loss indicates overfitting; if validation loss stops decreasing while training loss continues to fall, the model is memorizing rather than generalizing.

- How did the quality of your generated images change throughout the training process? ANS: In early in training, generated images look like structured noise with no recognizable features. By mid-training, rough shapes emerge. By late training, distinct digit features appear. This reflects the model progressively learning from coarse structure down to fine detail.
- Why do we need the time embedding in diffusion models? How does it help the model understand where it is in the denoising process? ANS: Without time embeddings, the model would have no way of knowing which denoising step it's at, and would try to apply the same correction regardless of noise level, which would be wrong. At $t=900$ (heavy noise) it needs to make large corrections, at $t=100$ (light noise) it needs subtle refinements. The sinusoidal time embedding gives the model a unique signal for each timestep so it can adjust its behavior accordingly.

1.10.4 4. CLIP Evaluation (20 points)

- What do the CLIP scores tell you about your generated images? Which images got the highest and lowest quality scores? ANS: CLIP scores measure how well the generated images match text descriptions like “a good handwritten digit” vs “a blurry image.” Higher scores on positive descriptions indicate perceptually convincing outputs. Digits with simpler structure like 1 and 7 typically score highest; complex digits like 8 tend to score lower.
- Develop a hypothesis explaining why certain images might be easier or harder for the model to generate convincingly. ANS: Digits with rotational symmetry or complex overlapping curves (8, 9, 6) require the model to learn more precise spatial relationships. Digits like 1 have fewer constraints and more margin for error. Additionally, class imbalance in training data can cause the model to generate some digits more reliably than others.
- How could CLIP scores be used to improve the diffusion model's generation process? Propose a specific technique. ANS: One specific technique is CLIP-guided diffusion; during the reverse process, at each denoising step we compute the gradient of the CLIP score with respect to the noisy image and nudge the image slightly in the direction that increases the score. This steers generation toward higher-quality outputs without retraining the model.

1.10.5 5. Practical Applications (20 points)

- How could this type of model be useful in the real world? ANS: Data augmentation for training other models, generating synthetic training data for rare classes (Although I fear for computational demand), image restoration and inpainting, creative tools for artists, and medical image synthesis where real data is scarce.
- What are the limitations of our current model? ANS: Limited to 28x28 grayscale images, slow inference (1000 denoising steps), no fine-grained style control beyond class label, and generation quality is heavily dependent on training time and GPU resources.
- If you were to continue developing this project, what three specific improvements would you make and why? ANS: First, implement DDIM sampling to reduce inference from 1000 steps to about 50 without quality loss. Second, add attention layers in the U-Net bottleneck to

improve long-range spatial coherence. Third, increase `down_chs` to (64, 128, 256) for more model capacity, which would capture finer detail at the cost of more training time.

1.10.6 Bonus Challenge (Extra 20 points)

Try one or more of these experiments: 1. If you were to continue developing this project, what three specific improvements would you make and why?

2. Modify the U-Net architecture (e.g., add more layers, increase channel dimensions) and train the model. How do these changes affect training time and generation quality?
3. CLIP-Guided Selection: Generate 10 samples of each image, use CLIP to evaluate them, and select the top 3 highest-quality examples of each. Analyze patterns in what CLIP considers “high quality.”
4. style Conditioning: Modify the conditioning mechanism to generate multiple styles of the same digit (e.g., slanted, thick, thin). Document your approach and results.

Deliverables: 1. A PDF copy of your notebook with - Complete code, outputs, and generated images
- Include all experiment results, training plots, and generated samples - CLIP evaluation scores of the images you generated - Answers and any interesting findings from the bonus challenges